Python Programming: Practical Programs with Solutions and Outputs

Units I–III — 217 Programs

List of Programs

Unit I: Computational Thinking and Python Fundamentals

No.	Program	Topic
1.1	Read Name and Age and Greet the User	Input, Output and Type Conversion
1.2	Sum of Two Numbers (int and float)	Input, Output and Type Conversion
1.3	Simple Interest	Input, Output and Type Conversion
1.4	Area and Perimeter of a Circle and a Rectangle	Input, Output and Type Conversion
1.5	Temperature Conversion (Celsius and Fahrenheit)	Input, Output and Type Conversion
1.6	Swap Two Numbers (Three Methods)	Input, Output and Type Conversion
1.7	Type Conversion Functions	Input, Output and Type Conversion
1.8	Formatted Output using f-strings, sep and end	Input, Output and Type Conversion
1.9	Shopping Bill with GST	Input, Output and Type Conversion
1.10	Employee Salary Slip	Input, Output and Type Conversion
1.11	Loan EMI Calculation	Input, Output and Type Conversion
1.12	Currency Conversion at a Fixed Rate	Input, Output and Type Conversion
1.13	Distance and Weight Unit Conversion	Input, Output and Type Conversion
1.14	Convert Seconds to Hours, Minutes and Seconds	Input, Output and Type Conversion
1.15	Convert Total Days to Years, Months and Days	Input, Output and Type Conversion
1.16	Total and Percentage of Marks in Five Subjects	Input, Output and Type Conversion
1.17	Average Speed of a Two-Part Journey	Input, Output and Type Conversion
1.18	Area of a Triangle using Heron’s Formula	Input, Output and Type Conversion
1.19	Cost of Painting a Wall	Input, Output and Type Conversion
1.20	Fuel Cost for a Trip	Input, Output and Type Conversion
1.21	ASCII Value of a Character and Character of a Code	Input, Output and Type Conversion
1.22	Decimal to Binary, Octal and Hexadecimal	Input, Output and Type Conversion
1.23	Arithmetic Operators	Arithmetic and Logical Operators
1.24	Quotient and Remainder	Arithmetic and Logical Operators
1.25	Check Even or Odd	Arithmetic and Logical Operators
1.26	Leap Year using Logical Operators	Arithmetic and Logical Operators
1.27	Largest of Three Numbers using ‘and’	Arithmetic and Logical Operators
1.28	Bitwise Operators	Arithmetic and Logical Operators
1.29	Membership and Identity Operators	Arithmetic and Logical Operators
1.30	Compound Assignment Operators	Arithmetic and Logical Operators
1.31	ATM Cash Withdrawal Note Denominations	Arithmetic and Logical Operators
1.32	Extract Digits of a Four-Digit Number	Arithmetic and Logical Operators
1.33	Check Divisibility by 3 and 5	Arithmetic and Logical Operators
1.34	Eligibility to Vote and Drive	Arithmetic and Logical Operators
1.35	Password Rule Check using and / or	Arithmetic and Logical Operators
1.36	Swap using XOR and Arithmetic Operators	Arithmetic and Logical Operators
1.37	Even/Odd and Power of Two using Bitwise AND	Arithmetic and Logical Operators
1.38	Profit or Loss Percentage	Arithmetic and Logical Operators
1.39	Discount and Final Price on MRP	Arithmetic and Logical Operators

No.	Program	Topic
1.40	Operator Precedence and Associativity	Expressions and Operator Precedence
1.41	Compound Interest	Expressions and Operator Precedence
1.42	Roots of a Quadratic Equation	Expressions and Operator Precedence
1.43	Distance between Two Points	Expressions and Operator Precedence
1.44	Body Mass Index (BMI)	Expressions and Operator Precedence
1.45	Simple vs Compound Interest (Yearly and Monthly)	Expressions and Operator Precedence
1.46	Volume and Surface Area of Cylinder, Sphere and Cone	Expressions and Operator Precedence
1.47	Sum of AP and GP Series using Formulas	Expressions and Operator Precedence
1.48	Slope, Midpoint and Equation of a Line	Expressions and Operator Precedence
1.49	Equations of Motion	Expressions and Operator Precedence
1.50	Evaluate a Polynomial at a Given x	Expressions and Operator Precedence
1.51	Hypotenuse using Pythagoras Theorem	Expressions and Operator Precedence
1.52	Percentage Increase or Decrease	Expressions and Operator Precedence
1.53	Kinetic and Potential Energy	Expressions and Operator Precedence
1.54	Factorial of a Number	Problem Solving (Algorithms)
1.55	Sum of Digits of a Number	Problem Solving (Algorithms)
1.56	Reverse a Number and Check Palindrome	Problem Solving (Algorithms)
1.57	Check whether a Number is Prime	Problem Solving (Algorithms)
1.58	GCD and LCM using Euclid's Algorithm	Problem Solving (Algorithms)
1.59	Fibonacci Series	Problem Solving (Algorithms)

Unit II: Control Structures and Strings

No.	Program	Topic
2.1	Positive, Negative or Zero	Conditional Statements
2.2	Even or Odd	Conditional Statements
2.3	Largest of Three Numbers (Nested if)	Conditional Statements
2.4	Grade Calculator (elif Ladder)	Conditional Statements
2.5	Leap Year Check	Conditional Statements
2.6	Vowel or Consonant	Conditional Statements
2.7	Simple Calculator using if-elif	Conditional Statements
2.8	Triangle Validity and Type	Conditional Statements
2.9	Electricity Bill (Slab Rates)	Conditional Statements
2.10	Character Type Check	Conditional Statements
2.11	Quadrant of a Point	Conditional Statements
2.12	BMI Calculator with Category	Conditional Statements
2.13	Triangle Classification by Angles	Conditional Statements
2.14	Cinema Ticket Price by Age and Day Type	Conditional Statements
2.15	Discount by Purchase Amount Slabs	Conditional Statements
2.16	Shipping Charge by Weight and Zone	Conditional Statements
2.17	ATM Withdrawal Validation	Conditional Statements
2.18	Admission Eligibility Check	Conditional Statements
2.19	Income Tax Calculation by Slabs	Conditional Statements
2.20	Nature of Roots of a Quadratic Equation	Conditional Statements

No.	Program	Topic
2.21	Sum of First n Natural Numbers (for and while)	Iterative Statements
2.22	Multiplication Table	Iterative Statements
2.23	Factorial of a Number	Iterative Statements
2.24	Fibonacci Series	Iterative Statements
2.25	Prime Check and Primes in a Range	Iterative Statements
2.26	Armstrong Number	Iterative Statements
2.27	Palindrome Number	Iterative Statements
2.28	Perfect Number	Iterative Statements
2.29	GCD and LCM of Two Numbers	Iterative Statements
2.30	Sum of Digits and Reverse of a Number	Iterative Statements
2.31	Demonstrating break, continue and pass	Iterative Statements
2.32	Number Guessing Game	Iterative Statements
2.33	Strong Number	Iterative Statements
2.34	Shopping Cart Total until 0 is Entered	Iterative Statements
2.35	PIN Verification with 3 Attempts	Iterative Statements
2.36	ATM Menu with Balance (Loop Driven)	Iterative Statements
2.37	Grade Sheet for n Students	Iterative Statements
2.38	Sum of Series (Harmonic and Alternating Power)	Iterative Statements
2.39	Binary to Decimal and Decimal to Binary	Iterative Statements
2.40	Prime Factors and Sum of Primes	Iterative Statements
2.41	Collatz Sequence	Iterative Statements
2.42	Number to Words (0-999)	Iterative Statements
2.43	Decimal to Roman Numeral Conversion	Iterative Statements
2.44	Compound Interest Year-by-Year Table	Iterative Statements
2.45	Loan EMI and Amortisation (First 6 Months)	Iterative Statements
2.46	Right-Angled Triangle of Stars	Pattern Generation Programs
2.47	Inverted Right Triangle	Pattern Generation Programs
2.48	Pyramid Pattern	Pattern Generation Programs
2.49	Diamond Pattern	Pattern Generation Programs
2.50	Number Triangle	Pattern Generation Programs
2.51	Floyd’s Triangle	Pattern Generation Programs
2.52	Pascal’s Triangle	Pattern Generation Programs
2.53	Alphabet Triangle Pattern	Pattern Generation Programs
2.54	Hollow Square Pattern	Pattern Generation Programs
2.55	Number Pyramid	Pattern Generation Programs
2.56	Inverted Number Pyramid	Pattern Generation Programs
2.57	Hollow Pyramid of Stars	Pattern Generation Programs
2.58	Hourglass Pattern of Stars	Pattern Generation Programs
2.59	Star X Pattern	Pattern Generation Programs
2.60	Butterfly Pattern	Pattern Generation Programs
2.61	0-1 Triangle Pattern	Pattern Generation Programs
2.62	String Methods Demonstration	String Manipulation Applications

No.	Program	Topic
2.63	Palindrome String	String Manipulation Applications
2.64	Count Vowels, Consonants, Digits and Spaces	String Manipulation Applications
2.65	Reverse Words in a Sentence	String Manipulation Applications
2.66	Character Frequency	String Manipulation Applications
2.67	Anagram Check	String Manipulation Applications
2.68	Remove Duplicates and Punctuation	String Manipulation Applications
2.69	String Formatting Techniques	String Manipulation Applications
2.70	Longest Word in a Sentence	String Manipulation Applications
2.71	Capitalise Each Word without title()	String Manipulation Applications
2.72	Toggle Case of a String	String Manipulation Applications
2.73	String Slicing Demonstration	String Manipulation Applications
2.74	Acronym Generator	String Manipulation Applications
2.75	Pangram Check	String Manipulation Applications
2.76	Mask Card Number Showing Last 4 Digits	String Manipulation Applications
2.77	Check Rotation of Strings	String Manipulation Applications
2.78	Word Replace and Count Occurrences	String Manipulation Applications
2.79	Run-Length String Compression	String Manipulation Applications
2.80	Caesar Cipher Encrypt and Decrypt	String Manipulation Applications
2.81	Simple Email Validation	String Manipulation Applications
2.82	Username Rules Validation	String Manipulation Applications
2.83	Password Strength Checker	String Manipulation Applications
2.84	Count Lines, Words and Characters of Multi-line Text	String Manipulation Applications

Unit III: Lists, Tuples, Sets and Functions

No.	Program	Topic
3.1	Basic List Operations	List Operations and Applications
3.2	Largest and Smallest Element without max/min	List Operations and Applications
3.3	Sum, Average and Second Largest	List Operations and Applications
3.4	Remove Duplicates and Count Frequency	List Operations and Applications
3.5	Linear Search	List Operations and Applications
3.6	Bubble Sort	List Operations and Applications
3.7	List Comprehension and Even/Odd Separation	List Operations and Applications
3.8	Rotate a List	List Operations and Applications
3.9	Matrix Addition, Multiplication and Transpose	List Operations and Applications
3.10	Student Marks Analysis using a List	List Operations and Applications
3.11	Menu-Driven Shopping List Manager	List Operations and Applications
3.12	Cumulative (Running) Sum of Daily Sales	List Operations and Applications
3.13	Temperature Readings - Days Above Average	List Operations and Applications
3.14	Count Positive, Negative and Zero Values	List Operations and Applications
3.15	Merge Two Sorted Lists without sort()	List Operations and Applications
3.16	Find All Pairs with a Given Sum	List Operations and Applications
3.17	Move All Zeros to the End	List Operations and Applications
3.18	Insertion Sort	List Operations and Applications

No.	Program	Topic
3.19	Selection Sort	List Operations and Applications
3.20	Stack Implementation using a List	List Operations and Applications
3.21	Queue Implementation using a List	List Operations and Applications
3.22	Tuple Creation, Indexing and Slicing	Tuple Manipulation Programs
3.23	Tuple Packing, Unpacking and Swapping	Tuple Manipulation Programs
3.24	List-Tuple Conversion and Immutability	Tuple Manipulation Programs
3.25	Sort Tuples by Second Element and Tuple Statistics	Tuple Manipulation Programs
3.26	Nested Tuple of Student Records	Tuple Manipulation Programs
3.27	Employee Records Sorted by Salary	Tuple Manipulation Programs
3.28	Product Catalogue - Total Stock Value	Tuple Manipulation Programs
3.29	Coordinates - Distance from Origin and Nearest Point	Tuple Manipulation Programs
3.30	Subject-wise Grades from a Tuple of Marks	Tuple Manipulation Programs
3.31	Find All Occurrences of an Element in a Tuple	Tuple Manipulation Programs
3.32	Zip Two Tuples into Pairs	Tuple Manipulation Programs
3.33	Date Tuple Validation and Formatting	Tuple Manipulation Programs
3.34	Set Operations using Methods and Operators	Set Operations and Membership Testing
3.35	Subset, Superset and Disjoint Sets	Set Operations and Membership Testing
3.36	Remove Duplicates and Common Elements using Sets	Set Operations and Membership Testing
3.37	Membership Testing and Frozenset	Set Operations and Membership Testing
3.38	Unique Words in a Sentence	Set Operations and Membership Testing
3.39	Attendance - Present and Absent Students	Set Operations and Membership Testing
3.40	Course Enrollment Analysis using Sets	Set Operations and Membership Testing
3.41	Unique Visitors from a Visitor Log	Set Operations and Membership Testing
3.42	Banned Word Filter using Set Membership	Set Operations and Membership Testing
3.43	Common Friends and Friend Suggestions	Set Operations and Membership Testing
3.44	Check Duplicates in a List using a Set	Set Operations and Membership Testing
3.45	Common Letters and Pangram Check using Sets	Set Operations and Membership Testing
3.46	Factorial using Recursion	Recursive Function Applications
3.47	Fibonacci Series using Recursion	Recursive Function Applications
3.48	Sum of Digits using Recursion	Recursive Function Applications
3.49	Power and GCD using Recursion	Recursive Function Applications
3.50	Reverse String and Palindrome Check using Recursion	Recursive Function Applications
3.51	Tower of Hanoi	Recursive Function Applications
3.52	Binary Search using Recursion	Recursive Function Applications
3.53	Decimal to Binary using Recursion	Recursive Function Applications
3.54	Sum and Maximum of a List using Recursion	Recursive Function Applications
3.55	Count Digits of a Number using Recursion	Recursive Function Applications
3.56	Print 1 to N and N to 1 using Recursion	Recursive Function Applications
3.57	Check if a List is Sorted using Recursion	Recursive Function Applications
3.58	Flatten a Nested List using Recursion	Recursive Function Applications
3.59	Permutations of a String using Recursion	Recursive Function Applications
3.60	nCr using Recursion (Pascal’s Rule)	Recursive Function Applications

No.	Program	Topic
3.61	Positional, Keyword and Default Arguments with Docstrings	User Defined Functions
3.62	Variable Length Arguments *args and **kwargs	User Defined Functions
3.63	Function Returning Multiple Values	User Defined Functions
3.64	Global and Local Scope and Closures	User Defined Functions
3.65	Lambda with map, filter, reduce and sorted	User Defined Functions
3.66	Prime Numbers using a Function	User Defined Functions
3.67	Menu-Driven Calculator using Functions	User Defined Functions
3.68	Menu-Driven Unit Converter using Functions	User Defined Functions
3.69	Mean, Median and Mode using Functions	User Defined Functions
3.70	Number Utility Functions - Armstrong, Palindrome, Perfect	User Defined Functions
3.71	Simple Interest and Loan EMI with Default Arguments	User Defined Functions
3.72	GST Bill Generator using Functions	User Defined Functions
3.73	Password Validation Function Returning Reasons	User Defined Functions
3.74	Plain vs Memoized Fibonacci - Function Call Count	User Defined Functions

Unit I: Computational Thinking and Python Fundamentals

Input, Output and Type Conversion

Program 1.1: Read Name and Age and Greet the User

Aim: To write a Python program that reads a user's name and age and displays a greeting along with the age after five years.

Program:

```
# Read the name (string) and age (converted to int)
name = input("Enter your name: ")
age = int(input("Enter your age: "))

print("Hello,", name + "! Welcome to Python.")
print("You are", age, "years old.")
print("After 5 years you will be", age + 5, "years old.")
```

Output:

```
Enter your name: Ravi
Enter your age: 18
Hello, Ravi! Welcome to Python.
You are 18 years old.
After 5 years you will be 23 years old.
```

Program 1.2: Sum of Two Numbers (int and float)

Aim: To write a Python program that reads two numbers and finds their sum using integer and floating-point conversion.

Program:

```
first = input("Enter first number: ")
second = input("Enter second number: ")

# Without conversion, + joins the two strings
print("String concatenation:", first + second)

# Integer addition
int_sum = int(first) + int(second)
print("Integer sum:", int_sum)

# Floating-point addition
float_sum = float(first) + float(second)
print("Float sum:", float_sum)
```

Output:

```
Enter first number: 25
Enter second number: 17
String concatenation: 2517
Integer sum: 42
Float sum: 42.0
```

Program 1.3: Simple Interest

Aim: To write a Python program that calculates simple interest and the total amount for a given principal, rate and time.

Program:

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter rate of interest (per annum): "))
time = float(input("Enter time (in years): "))

# Simple interest formula
simple_interest = (principal * time * rate) / 100
amount = principal + simple_interest

print("Simple Interest =", simple_interest)
print("Total Amount =", amount)
```

Output:

```
Enter principal amount: 10000
Enter rate of interest (per annum): 8.5
Enter time (in years): 3
Simple Interest = 2550.0
Total Amount = 12550.0
```

Program 1.4: Area and Perimeter of a Circle and a Rectangle

Aim: To write a Python program that computes the area and perimeter of a circle and of a rectangle.

Program:

```
import math

# Circle
radius = float(input("Enter radius of the circle: "))
circle_area = math.pi * radius * radius
circumference = 2 * math.pi * radius
print("Area of circle =", round(circle_area, 2))
print("Circumference of circle =", round(circumference, 2))

# Rectangle
length = float(input("Enter length of the rectangle: "))
breadth = float(input("Enter breadth of the rectangle: "))
rect_area = length * breadth
rect_perimeter = 2 * (length + breadth)
print("Area of rectangle =", rect_area)
print("Perimeter of rectangle =", rect_perimeter)
```

Output:

```
Enter radius of the circle: 7
Area of circle = 153.94
Circumference of circle = 43.98
Enter length of the rectangle: 12
Enter breadth of the rectangle: 5
```

```
Area of rectangle = 60.0
Perimeter of rectangle = 34.0
```

Program 1.5: Temperature Conversion (Celsius and Fahrenheit)

Aim: To write a Python program that converts a temperature from Celsius to Fahrenheit and from Fahrenheit to Celsius.

Program:

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print(f"{celsius} C = {fahrenheit:.2f} F")

fahrenheit = float(input("Enter temperature in Fahrenheit: "))
celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit} F = {celsius:.2f} C")
```

Output:

```
Enter temperature in Celsius: 37
37.0 C = 98.60 F
Enter temperature in Fahrenheit: 98.6
98.6 F = 37.00 C
```

Program 1.6: Swap Two Numbers (Three Methods)

Aim: To write a Python program that swaps two numbers using a temporary variable, arithmetic operators and tuple assignment.

Program:

```
a = int(input("Enter value of a: "))
b = int(input("Enter value of b: "))
print("Before swapping: a =", a, ", b =", b)

# Method 1: using a temporary variable
temp = a
a = b
b = temp
print("After swap using temp variable: a =", a, ", b =", b)

# Method 2: using arithmetic operators (swaps them back)
a = a + b
b = a - b
a = a - b
print("After swap using + and -: a =", a, ", b =", b)

# Method 3: using tuple assignment (Pythonic way)
a, b = b, a
print("After swap using tuple assignment: a =", a, ", b =", b)
```

Output:

```
Enter value of a: 10
Enter value of b: 20
Before swapping: a = 10 , b = 20
```

```
After swap using temp variable: a = 20 , b = 10
After swap using + and -: a = 10 , b = 20
After swap using tuple assignment: a = 20 , b = 10
```

Program 1.7: Type Conversion Functions

Aim: To write a Python program that demonstrates type conversion using int(), float(), str(), bool(), complex(), bin(), oct() and hex().

Program:

```
number = int(input("Enter an integer: "))

print("Value:", number, "Type:", type(number))
print("float(number):", float(number))
print("str(number):", str(number), "Type:", type(str(number)))
print("bool(number):", bool(number))
print("bool(0):", bool(0))
print("complex(number):", complex(number))
print("complex(3, 4):", complex(3, 4))
print("int(7.9) truncates to:", int(7.9))

# Number system conversions
print("Binary:", bin(number))
print("Octal:", oct(number))
print("Hexadecimal:", hex(number))

# Converting strings in other bases to decimal
print('int("101", 2) =', int("101", 2))
print('int("17", 8) =', int("17", 8))
print('int("FF", 16) =', int("FF", 16))
```

Output:

```
Enter an integer: 45
Value: 45 Type: <class 'int'>
float(number): 45.0
str(number): 45 Type: <class 'str'>
bool(number): True
bool(0): False
complex(number): (45+0j)
complex(3, 4): (3+4j)
int(7.9) truncates to: 7
Binary: 0b101101
Octal: 0o55
Hexadecimal: 0x2d
int("101", 2) = 5
int("17", 8) = 15
int("FF", 16) = 255
```

Program 1.8: Formatted Output using f-strings, sep and end

Aim: To write a Python program that displays formatted output using f-strings, format specifiers and the sep and end parameters of print().

Program:

```
name = input("Enter student name: ")
marks = float(input("Enter marks obtained (out of 600): "))
percentage = marks / 600 * 100

# sep and end parameters
print("Python", "is", "fun", sep="-")
print("No newline here", end=" | ")
print("printed on the same line")

# f-string formatting
print(f"Name: {name}")
print(f"Marks: {marks:.1f}")
print(f"Percentage: {percentage:.2f}%")
print(f"[{name:<10}] left aligned")
print(f"[{name:>10}] right aligned")
print(f"[{name:^10}] centred")
print(f"Large number with commas: {1234567:,}")

# format() method (older style)
print("Name: {} scored {:.2f}%".format(name, percentage))
```

Output:

```
Enter student name: Anitha
Enter marks obtained (out of 600): 512
Python-is-fun
No newline here | printed on the same line
Name: Anitha
Marks: 512.0
Percentage: 85.33%
[Anitha    ] left aligned
[    Anitha] right aligned
[  Anitha  ] centred
Large number with commas: 1,234,567
Name: Anitha scored 85.33%
```

Program 1.9: Shopping Bill with GST

Aim: To write a Python program that prepares a shopping bill for an item by calculating the amount, GST and the total payable.

Program:

```
# Read item details
item = input("Enter item name: ")
price = float(input("Enter unit price (Rs): "))
quantity = int(input("Enter quantity: "))
gst_rate = float(input("Enter GST rate (%): "))

# Calculate the bill
amount = price * quantity
gst = amount * gst_rate / 100
total = amount + gst

print("---------- BILL ----------")
print(f"Item       : {item}")
```

```
print(f"Price x Qty: {price:.2f} x {quantity}")
print(f"Amount     : Rs {amount:.2f}")
print(f"GST @ {gst_rate}% : Rs {gst:.2f}")
print(f"Total      : Rs {total:.2f}")
```

Output:

```
Enter item name: Notebook
Enter unit price (Rs): 45.50
Enter quantity: 12
Enter GST rate (%): 18
---------- BILL ----------
Item       : Notebook
Price x Qty: 45.50 x 12
Amount     : Rs 546.00
GST @ 18.0% : Rs 98.28
Total      : Rs 644.28
```

Program 1.10: Employee Salary Slip

Aim: To write a Python program that generates a salary slip showing basic pay, DA, HRA, PF, gross salary and net salary.

Program:

```
# Read employee details
name = input("Enter employee name: ")
emp_id = input("Enter employee ID: ")
basic = float(input("Enter basic pay (Rs): "))

# Allowances and deductions as percentages of basic pay
da = basic * 42 / 100
hra = basic * 20 / 100
pf = basic * 12 / 100
gross = basic + da + hra
net = gross - pf

print("========= SALARY SLIP =========")
print(f"Name : {name}    ID : {emp_id}")
print(f"Basic Pay     : {basic:>10.2f}")
print(f"DA  (42%)     : {da:>10.2f}")
print(f"HRA (20%)     : {hra:>10.2f}")
print(f"Gross Salary  : {gross:>10.2f}")
print(f"PF  (12%)     : {pf:>10.2f}")
print(f"Net Salary    : {net:>10.2f}")
```

Output:

```
Enter employee name: Ravi Kumar
Enter employee ID: E1024
Enter basic pay (Rs): 35000
========= SALARY SLIP =========
Name : Ravi Kumar    ID : E1024
Basic Pay     :   35000.00
DA  (42%)     :   14700.00
HRA (20%)     :    7000.00
Gross Salary  :   56700.00
```

```
PF  (12%)     :    4200.00
Net Salary    :   52500.00
```

Program 1.11: Loan EMI Calculation

Aim: To write a Python program that calculates the monthly EMI, total payment and total interest for a loan.

Program:

```
principal = float(input("Enter loan amount (Rs): "))
annual_rate = float(input("Enter annual interest rate (%): "))
years = int(input("Enter loan tenure (years): "))

r = annual_rate / (12 * 100)   # monthly interest rate
n = years * 12                 # number of monthly instalments
factor = (1 + r) ** n

emi = principal * r * factor / (factor - 1)
total_payment = emi * n
total_interest = total_payment - principal

print(f"Monthly EMI      : Rs {emi:,.2f}")
print(f"Number of months : {n}")
print(f"Total payment    : Rs {total_payment:,.2f}")
print(f"Total interest   : Rs {total_interest:,.2f}")
```

Output:

```
Enter loan amount (Rs): 500000
Enter annual interest rate (%): 9.5
Enter loan tenure (years): 5
Monthly EMI      : Rs 10,500.93
Number of months : 60
Total payment    : Rs 630,055.84
Total interest   : Rs 130,055.84
```

Program 1.12: Currency Conversion at a Fixed Rate

Aim: To write a Python program that converts an amount in Indian Rupees into US Dollars, Euros and British Pounds at fixed exchange rates.

Program:

```
# Fixed exchange rates (Rupees for 1 unit of foreign currency)
USD_RATE = 83.25
EUR_RATE = 90.10
GBP_RATE = 105.40

rupees = float(input("Enter amount in INR: "))

usd = rupees / USD_RATE
eur = rupees / EUR_RATE
gbp = rupees / GBP_RATE

print(f"Rs {rupees:.2f} is equal to:")
```

```
print(f"  US Dollars    : $ {usd:.2f}")
print(f"  Euros         : EUR {eur:.2f}")
print(f"  British Pounds: GBP {gbp:.2f}")
```

Output:

```
Enter amount in INR: 25000
Rs 25000.00 is equal to:
  US Dollars    : $ 300.30
  Euros         : EUR 277.47
  British Pounds: GBP 237.19
```

Program 1.13: Distance and Weight Unit Conversion

Aim: To write a Python program that converts kilometres to miles, miles to kilometres, kilograms to pounds and pounds to kilograms.

Program:

```
KM_TO_MILES = 0.621371
MILES_TO_KM = 1.609344
KG_TO_LB = 2.20462
LB_TO_KG = 0.453592

km = float(input("Enter distance in kilometres: "))
miles = float(input("Enter distance in miles: "))
kg = float(input("Enter weight in kilograms: "))
pounds = float(input("Enter weight in pounds: "))

print(f"{km} km = {km * KM_TO_MILES:.3f} miles")
print(f"{miles} miles = {miles * MILES_TO_KM:.3f} km")
print(f"{kg} kg = {kg * KG_TO_LB:.3f} lb")
print(f"{pounds} lb = {pounds * LB_TO_KG:.3f} kg")
```

Output:

```
Enter distance in kilometres: 42.195
Enter distance in miles: 26.2
Enter weight in kilograms: 65
Enter weight in pounds: 150
42.195 km = 26.219 miles
26.2 miles = 42.165 km
65.0 kg = 143.300 lb
150.0 lb = 68.039 kg
```

Program 1.14: Convert Seconds to Hours, Minutes and Seconds

Aim: To write a Python program that converts a given number of seconds into hours, minutes and seconds in h:m:s form.

Program:

```
total_seconds = int(input("Enter time in seconds: "))

hours = total_seconds // 3600
minutes = (total_seconds % 3600) // 60
seconds = total_seconds % 60
```

```
print(f"{total_seconds} seconds = {hours} hours, {minutes} minutes, {seconds} seconds")
print(f"Time in h:m:s format: {hours}:{minutes:02d}:{seconds:02d}")
```

Output:

```
Enter time in seconds: 9876
9876 seconds = 2 hours, 44 minutes, 36 seconds
Time in h:m:s format: 2:44:36
```

Program 1.15: Convert Total Days to Years, Months and Days

Aim: To write a Python program that converts a total number of days into years, months and days (1 year = 365 days, 1 month = 30 days).

Program:

```
total_days = int(input("Enter total number of days: "))

years = total_days // 365
remaining = total_days % 365
months = remaining // 30
days = remaining % 30

print(f"{total_days} days = {years} years, {months} months and {days} days")
```

Output:

```
Enter total number of days: 1000
1000 days = 2 years, 9 months and 0 days
```

Program 1.16: Total and Percentage of Marks in Five Subjects

Aim: To write a Python program that reads marks of five subjects and displays the total, average and percentage.

Program:

```
name = input("Enter student name: ")
m1 = float(input("Enter marks in Mathematics: "))
m2 = float(input("Enter marks in Physics: "))
m3 = float(input("Enter marks in Chemistry: "))
m4 = float(input("Enter marks in English: "))
m5 = float(input("Enter marks in Programming: "))

total = m1 + m2 + m3 + m4 + m5
average = total / 5
percentage = total / 500 * 100

print(f"Student    : {name}")
print(f"Total      : {total} / 500")
print(f"Average    : {average:.2f}")
print(f"Percentage : {percentage:.2f}%")
```

Output:

```
Enter student name: Anitha
Enter marks in Mathematics: 88
Enter marks in Physics: 76
Enter marks in Chemistry: 91
Enter marks in English: 69.5
Enter marks in Programming: 95
Student    : Anitha
Total      : 419.5 / 500
Average    : 83.90
Percentage : 83.90%
```

Program 1.17: Average Speed of a Two-Part Journey

Aim: To write a Python program that calculates the speed for each part of a journey and the average speed for the whole trip.

Program:

```
d1 = float(input("Enter distance of first part (km): "))
t1 = float(input("Enter time for first part (hours): "))
d2 = float(input("Enter distance of second part (km): "))
t2 = float(input("Enter time for second part (hours): "))

speed1 = d1 / t1
speed2 = d2 / t2
average_speed = (d1 + d2) / (t1 + t2)

print(f"Speed in first part  : {speed1:.2f} km/h")
print(f"Speed in second part : {speed2:.2f} km/h")
print(f"Average speed        : {average_speed:.2f} km/h")
```

Output:

```
Enter distance of first part (km): 120
Enter time for first part (hours): 2
Enter distance of second part (km): 90
Enter time for second part (hours): 2.5
Speed in first part  : 60.00 km/h
Speed in second part : 36.00 km/h
Average speed        : 46.67 km/h
```

Program 1.18: Area of a Triangle using Heron's Formula

Aim: To write a Python program that finds the perimeter and area of a triangle from its three sides using Heron's formula.

Program:

```
a = float(input("Enter side a: "))
b = float(input("Enter side b: "))
c = float(input("Enter side c: "))

perimeter = a + b + c
s = perimeter / 2
area = (s * (s - a) * (s - b) * (s - c)) ** 0.5
```

```
print(f"Perimeter      : {perimeter:.2f}")
print(f"Semi-perimeter : {s:.2f}")
print(f"Area           : {area:.2f} square units")
```

Output:

```
Enter side a: 7
Enter side b: 8
Enter side c: 9
Perimeter      : 24.00
Semi-perimeter : 12.00
Area           : 26.83 square units
```

Program 1.19: Cost of Painting a Wall

Aim: To write a Python program that calculates the paintable area of a wall (excluding a door and a window) and the cost of painting it.

Program:

```
length = float(input("Enter wall length (ft): "))
height = float(input("Enter wall height (ft): "))
door_w = float(input("Enter door width (ft): "))
door_h = float(input("Enter door height (ft): "))
window_area = float(input("Enter window area (sq ft): "))
rate = float(input("Enter painting rate (Rs per sq ft): "))

wall_area = length * height
door_area = door_w * door_h
paint_area = wall_area - door_area - window_area
cost = paint_area * rate

print(f"Wall area      : {wall_area:.2f} sq ft")
print(f"Paintable area : {paint_area:.2f} sq ft")
print(f"Painting cost  : Rs {cost:.2f}")
```

Output:

```
Enter wall length (ft): 20
Enter wall height (ft): 10
Enter door width (ft): 3
Enter door height (ft): 7
Enter window area (sq ft): 12
Enter painting rate (Rs per sq ft): 18.5
Wall area      : 200.00 sq ft
Paintable area : 167.00 sq ft
Painting cost  : Rs 3089.50
```

Program 1.20: Fuel Cost for a Trip

Aim: To write a Python program that calculates the fuel required and the fuel cost for a round trip, and the cost per person.

Program:

```
one_way = float(input("Enter one-way distance (km): "))
mileage = float(input("Enter vehicle mileage (km per litre): "))
```

```
price = float(input("Enter fuel price (Rs per litre): "))
persons = int(input("Enter number of travellers: "))

total_distance = one_way * 2
fuel_needed = total_distance / mileage
fuel_cost = fuel_needed * price
cost_per_person = fuel_cost / persons

print(f"Total distance  : {total_distance:.1f} km")
print(f"Fuel needed     : {fuel_needed:.2f} litres")
print(f"Total fuel cost : Rs {fuel_cost:.2f}")
print(f"Cost per person : Rs {cost_per_person:.2f}")
```

Output:

```
Enter one-way distance (km): 275
Enter vehicle mileage (km per litre): 18
Enter fuel price (Rs per litre): 102.5
Enter number of travellers: 4
Total distance  : 550.0 km
Fuel needed     : 30.56 litres
Total fuel cost : Rs 3131.94
Cost per person : Rs 782.99
```

Program 1.21: ASCII Value of a Character and Character of a Code

Aim: To write a Python program that displays the ASCII (Unicode) value of a character and the character for a given code using ord() and chr().

Program:

```
ch = input("Enter a character: ")
code = int(input("Enter an ASCII code (32-126): "))

print(f"ASCII value of '{ch}' is {ord(ch)}")
print(f"Character with code {code} is '{chr(code)}'")
print(f"Next character after '{ch}' is '{chr(ord(ch) + 1)}'")
# Letters: lower case code = upper case code + 32
print(f"'{ch}' with case swapped (code +/- 32): '{ch.swapcase()}' (code {ord(ch.swapcase(
))})")
```

Output:

```
Enter a character: G
Enter an ASCII code (32-126): 97
ASCII value of 'G' is 71
Character with code 97 is 'a'
Next character after 'G' is 'H'
'G' with case swapped (code +/- 32): 'g' (code 103)
```

Program 1.22: Decimal to Binary, Octal and Hexadecimal

Aim: To write a Python program that converts a decimal number into binary, octal and hexadecimal, and converts a binary string back to decimal.

Program:

```
number = int(input("Enter a decimal number: "))
binary_text = input("Enter a binary number: ")

print(f"Decimal     : {number}")
print(f"Binary      : {bin(number)}  (without prefix: {format(number, 'b')})")
print(f"Octal       : {oct(number)}")
print(f"Hexadecimal : {hex(number)}  (upper case: {format(number, 'X')})")
print(f"Binary {binary_text} in decimal = {int(binary_text, 2)}")
```

Output:

```
Enter a decimal number: 255
Enter a binary number: 101101
Decimal     : 255
Binary      : 0b11111111  (without prefix: 11111111)
Octal       : 0o377
Hexadecimal : 0xff  (upper case: FF)
Binary 101101 in decimal = 45
```

Arithmetic and Logical Operators

Program 1.23: Arithmetic Operators

Aim: To write a Python program that performs all arithmetic operations on two numbers.

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Addition       :", a, "+", b, "=", a + b)
print("Subtraction    :", a, "-", b, "=", a - b)
print("Multiplication :", a, "*", b, "=", a * b)
print("Division       :", a, "/", b, "=", a / b)
print("Floor division :", a, "//", b, "=", a // b)
print("Modulus        :", a, "%", b, "=", a % b)
print("Exponent       :", a, "**", b, "=", a ** b)
```

Output:

```
Enter first number: 17
Enter second number: 5
Addition       : 17 + 5 = 22
Subtraction    : 17 - 5 = 12
Multiplication : 17 * 5 = 85
Division       : 17 / 5 = 3.4
Floor division : 17 // 5 = 3
Modulus        : 17 % 5 = 2
Exponent       : 17 ** 5 = 1419857
```

Program 1.24: Quotient and Remainder

Aim: To write a Python program that finds the quotient and remainder of two integers.

Program:

```
dividend = int(input("Enter dividend: "))
divisor = int(input("Enter divisor: "))

quotient = dividend // divisor
remainder = dividend % divisor
print("Quotient =", quotient)
print("Remainder =", remainder)

# Using the built-in divmod() function
print("Using divmod():", divmod(dividend, divisor))

# Verification
print("Check:", divisor, "*", quotient, "+", remainder, "=",
      divisor * quotient + remainder)
```

Output:

```
Enter dividend: 47
Enter divisor: 6
Quotient = 7
Remainder = 5
Using divmod(): (7, 5)
Check: 6 * 7 + 5 = 47
```

Program 1.25: Check Even or Odd

Aim: To write a Python program that checks whether a given number is even or odd.

Program:

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print(number, "is an Even number")
else:
    print(number, "is an Odd number")
```

Output:

```
Enter a number: 27
27 is an Odd number
```

Program 1.26: Leap Year using Logical Operators

Aim: To write a Python program that checks whether a given year is a leap year using logical operators.

Program:

```
year = int(input("Enter a year: "))

# Leap year rule combined with logical operators
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a Leap Year")
else:
    print(year, "is not a Leap Year")
```

Output:

```
Enter a year: 2024
2024 is a Leap Year
```

Program 1.27: Largest of Three Numbers using 'and'

Aim: To write a Python program that finds the largest of three numbers using relational and logical operators.

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c

print("Largest number is", largest)
print("Verified using max():", max(a, b, c))
```

Output:

```
Enter first number: 45
Enter second number: 89
Enter third number: 32
Largest number is 89
Verified using max(): 89
```

Program 1.28: Bitwise Operators

Aim: To write a Python program that demonstrates the bitwise operators &, |, ^, ~, << and >>.

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("a =", a, "binary:", bin(a))
print("b =", b, "binary:", bin(b))
print("a & b  (AND)         =", a & b, bin(a & b))
print("a | b  (OR)          =", a | b, bin(a | b))
print("a ^ b  (XOR)         =", a ^ b, bin(a ^ b))
print("~a     (NOT)         =", ~a)
print("a << 2 (Left shift)  =", a << 2)
print("a >> 2 (Right shift) =", a >> 2)
```

Output:

```
Enter first number: 12
Enter second number: 10
a = 12 binary: 0b1100
b = 10 binary: 0b1010
```

```
a & b  (AND)         = 8 0b1000
a | b  (OR)          = 14 0b1110
a ^ b  (XOR)         = 6 0b110
~a     (NOT)         = -13
a << 2 (Left shift)  = 48
a >> 2 (Right shift) = 3
```

Program 1.29: Membership and Identity Operators

Aim: To write a Python program that demonstrates membership (in, not in) and identity (is, is not) operators.

Program:

```
text = input("Enter a sentence: ")
word = input("Enter a word to search: ")

# Membership operators
print("Is word in sentence?", word in text)
print("Is word not in sentence?", word not in text)

vowels = ["a", "e", "i", "o", "u"]
print("'e' in vowels:", "e" in vowels)
print("'z' not in vowels:", "z" not in vowels)

# Identity operators
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = list1
print("list1 == list2:", list1 == list2)
print("list1 is list2:", list1 is list2)
print("list1 is list3:", list1 is list3)
print("list1 is not list2:", list1 is not list2)
```

Output:

```
Enter a sentence: Python is easy to learn
Enter a word to search: easy
Is word in sentence? True
Is word not in sentence? False
'e' in vowels: True
'z' not in vowels: True
list1 == list2: True
list1 is list2: False
list1 is list3: True
list1 is not list2: True
```

Program 1.30: Compound Assignment Operators

Aim: To write a Python program that demonstrates compound (augmented) assignment operators.

Program:

```
x = int(input("Enter initial value of x: "))
print("Initial x =", x)

x += 5
```

```
print("After x += 5  :", x)
x -= 3
print("After x -= 3  :", x)
x *= 4
print("After x *= 4  :", x)
x //= 3
print("After x //= 3 :", x)
x %= 7
print("After x %= 7  :", x)
x **= 3
print("After x **= 3 :", x)
x /= 2
print("After x /= 2  :", x)
```

Output:

```
Enter initial value of x: 10
Initial x = 10
After x += 5  : 15
After x -= 3  : 12
After x *= 4  : 48
After x //= 3 : 16
After x %= 7  : 2
After x **= 3 : 8
After x /= 2  : 4.0
```

Program 1.31: ATM Cash Withdrawal Note Denominations

Aim: To write a Python program that finds the minimum number of 500, 200 and 100 rupee notes an ATM dispenses for a withdrawal amount.

Program:

```
amount = int(input("Enter withdrawal amount (multiple of 100): "))

notes_500 = amount // 500
balance = amount % 500
notes_200 = balance // 200
balance = balance % 200
notes_100 = balance // 100

print(f"Amount requested : Rs {amount}")
print(f"Rs 500 x {notes_500} = {notes_500 * 500}")
print(f"Rs 200 x {notes_200} = {notes_200 * 200}")
print(f"Rs 100 x {notes_100} = {notes_100 * 100}")
print(f"Total notes      : {notes_500 + notes_200 + notes_100}")
```

Output:

```
Enter withdrawal amount (multiple of 100): 7800
Amount requested : Rs 7800
Rs 500 x 15 = 7500
Rs 200 x 1 = 200
Rs 100 x 1 = 100
Total notes      : 17
```

Program 1.32: Extract Digits of a Four-Digit Number

Aim: To write a Python program that extracts each digit of a four-digit number and finds the sum of the digits and the sum of the first and last digits.

Program:

```
n = int(input("Enter a four-digit number: "))

thousands = n // 1000
hundreds = n // 100 % 10
tens = n // 10 % 10
units = n % 10

print(f"Digits : {thousands} {hundreds} {tens} {units}")
print(f"Sum of all digits            : {thousands + hundreds + tens + units}")
print(f"Sum of first and last digits : {thousands + units}")
print(f"Reversed number              : {units * 1000 + tens * 100 + hundreds * 10 + thous
ands}")
```

Output:

```
Enter a four-digit number: 4728
Digits : 4 7 2 8
Sum of all digits            : 21
Sum of first and last digits : 12
Reversed number              : 8274
```

Program 1.33: Check Divisibility by 3 and 5

Aim: To write a Python program that checks whether a number is divisible by 3, by 5, by both and by either, printing True or False.

Program:

```
n = int(input("Enter a number: "))

div3 = n % 3 == 0
div5 = n % 5 == 0

print("Divisible by 3        :", div3)
print("Divisible by 5        :", div5)
print("Divisible by both     :", div3 and div5)
print("Divisible by either   :", div3 or div5)
print("Divisible by neither  :", not (div3 or div5))
```

Output:

```
Enter a number: 45
Divisible by 3        : True
Divisible by 5        : True
Divisible by both     : True
Divisible by either   : True
Divisible by neither  : False
```

Program 1.34: Eligibility to Vote and Drive

Aim: To write a Python program that checks a person's eligibility to vote and to get a driving licence using relational and logical operators.

Program:

```
name = input("Enter name: ")
age = int(input("Enter age: "))
citizen = input("Indian citizen? (yes/no): ").strip().lower() == "yes"
passed_test = input("Passed driving test? (yes/no): ").strip().lower() == "yes"

can_vote = age >= 18 and citizen
can_drive = age >= 18 and passed_test

print(f"{name} can vote  : {can_vote}")
print(f"{name} can drive : {can_drive}")
print("Status:", "Eligible for both" if can_vote and can_drive else "Not eligible for bot
h")
```

Output:

```
Enter name: Kiran
Enter age: 19
Indian citizen? (yes/no): yes
Passed driving test? (yes/no): no
Kiran can vote  : True
Kiran can drive : False
Status: Not eligible for both
```

Program 1.35: Password Rule Check using and / or

Aim: To write a Python program that checks whether a password follows the rules using logical operators and prints True or False.

Program:

```
username = input("Enter username: ")
password = input("Enter password: ")

length_ok = 8 <= len(password) <= 16
mixed_ok = not password.isalpha() and not password.isdigit()
differs_ok = password.lower() != username.lower()

print("Length 8-16            :", length_ok)
print("Mix of letters & others:", mixed_ok)
print("Different from username:", differs_ok)
print("Password is valid      :", length_ok and mixed_ok and differs_ok)
print("Password is weak       :", not length_ok or not mixed_ok)
```

Output:

```
Enter username: student01
Enter password: lab@2026py
Length 8-16            : True
Mix of letters & others: True
Different from username: True
```

```
Password is valid      : True
Password is weak       : False
```

Program 1.36: Swap using XOR and Arithmetic Operators

Aim: To write a Python program that swaps two integers without a third variable, first with arithmetic operators and then with the XOR operator.

Program:

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
print(f"Before swapping : a = {a}, b = {b}")

# Method 1: arithmetic operators
a = a + b
b = a - b
a = a - b
print(f"After arithmetic swap : a = {a}, b = {b}")

# Method 2: bitwise XOR (swaps them back)
a = a ^ b
b = a ^ b
a = a ^ b
print(f"After XOR swap        : a = {a}, b = {b}")
```

Output:

```
Enter a: 25
Enter b: 40
Before swapping : a = 25, b = 40
After arithmetic swap : a = 40, b = 25
After XOR swap        : a = 25, b = 40
```

Program 1.37: Even/Odd and Power of Two using Bitwise AND

Aim: To write a Python program that checks whether a number is even or odd and whether it is a power of two using the bitwise AND operator.

Program:

```
n = int(input("Enter a positive integer: "))

print(f"Binary of {n} : {bin(n)}")
print(f"{n} is", "Even" if n & 1 == 0 else "Odd")

is_power_of_two = n > 0 and n & (n - 1) == 0
print(f"{n} is a power of two:", is_power_of_two)
```

Output:

```
Enter a positive integer: 64
Binary of 64 : 0b1000000
64 is Even
64 is a power of two: True
```

Program 1.38: Profit or Loss Percentage

Aim: To write a Python program that finds whether a sale results in profit or loss and calculates the amount and percentage.

Program:

```
cost_price = float(input("Enter cost price (Rs): "))
selling_price = float(input("Enter selling price (Rs): "))

difference = selling_price - cost_price
percent = abs(difference) / cost_price * 100
result = "Profit" if difference > 0 else "Loss" if difference < 0 else "No profit no loss
"

print(f"Result     : {result}")
print(f"Amount     : Rs {abs(difference):.2f}")
print(f"Percentage : {percent:.2f}%")
```

Output:

```
Enter cost price (Rs): 1250
Enter selling price (Rs): 1100
Result     : Loss
Amount     : Rs 150.00
Percentage : 12.00%
```

Program 1.39: Discount and Final Price on MRP

Aim: To write a Python program that calculates the discount amount and the final price of a product, with an extra 5% for orders above Rs 5000.

Program:

```
mrp = float(input("Enter MRP (Rs): "))
discount_pct = float(input("Enter discount (%): "))

discount = mrp * discount_pct / 100
price = mrp - discount

# Extra 5% only when price exceeds 5000 (True acts as 1, False as 0)
extra_pct = (price > 5000) * 5
extra = price * extra_pct / 100
final_price = price - extra

print(f"MRP              : Rs {mrp:.2f}")
print(f"Discount ({discount_pct}%) : Rs {discount:.2f}")
print(f"Extra discount ({extra_pct}%): Rs {extra:.2f}")
print(f"Final price      : Rs {final_price:.2f}")
print(f"You saved        : Rs {mrp - final_price:.2f}")
```

Output:

```
Enter MRP (Rs): 7999
Enter discount (%): 20
MRP              : Rs 7999.00
Discount (20.0%) : Rs 1599.80
```

```
Extra discount (5%): Rs 319.96
Final price      : Rs 6079.24
You saved        : Rs 1919.76
```

Expressions and Operator Precedence

Program 1.40: Operator Precedence and Associativity

Aim: To write a Python program that evaluates expressions to show operator precedence and associativity.

Program:

```
# ** first, then * and / (left to right), then +
print("2 + 3 * 4 ** 2 / 8 =", 2 + 3 * 4 ** 2 / 8)

# Parentheses change the order
print("(2 + 3) * 4 =", (2 + 3) * 4)
print("2 + 3 * 4 =", 2 + 3 * 4)

# ** is right-associative: 2 ** (3 ** 2)
print("2 ** 3 ** 2 =", 2 ** 3 ** 2)
print("(2 ** 3) ** 2 =", (2 ** 3) ** 2)

# ** has higher precedence than unary minus: -(3 ** 2)
print("-3 ** 2 =", -3 ** 2)
print("(-3) ** 2 =", (-3) ** 2)

# Left-to-right associativity for / and //
print("100 / 10 / 5 =", 100 / 10 / 5)
print("17 // 5 * 2 =", 17 // 5 * 2)

# Relational before logical; 'and' before 'or'
print("5 > 3 and 2 > 4 or 10 == 10 =", 5 > 3 and 2 > 4 or 10 == 10)
print("not 5 > 3 =", not 5 > 3)
```

Output:

```
2 + 3 * 4 ** 2 / 8 = 8.0
(2 + 3) * 4 = 20
2 + 3 * 4 = 14
2 ** 3 ** 2 = 512
(2 ** 3) ** 2 = 64
-3 ** 2 = -9
(-3) ** 2 = 9
100 / 10 / 5 = 2.0
17 // 5 * 2 = 6
5 > 3 and 2 > 4 or 10 == 10 = True
not 5 > 3 = False
```

Program 1.41: Compound Interest

Aim: To write a Python program that calculates compound interest for a given principal, rate, time and compounding frequency.

Program:

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter annual rate of interest: "))
time = float(input("Enter time (in years): "))
n = int(input("Enter number of times interest is compounded per year: "))

amount = principal * (1 + rate / (100 * n)) ** (n * time)
compound_interest = amount - principal

print(f"Amount = {amount:.2f}")
print(f"Compound Interest = {compound_interest:.2f}")
```

Output:

```
Enter principal amount: 10000
Enter annual rate of interest: 10
Enter time (in years): 2
Enter number of times interest is compounded per year: 1
Amount = 12100.00
Compound Interest = 2100.00
```

Program 1.42: Roots of a Quadratic Equation

Aim: To write a Python program that finds the roots of a quadratic equation ax^2 + bx + c = 0 using the discriminant.

Program:

```
import math
import cmath


def solve_quadratic(a, b, c):
    """Print the roots of ax^2 + bx + c = 0."""
    d = b * b - 4 * a * c
    print("Discriminant =", d)
    if d > 0:
        root1 = (-b + math.sqrt(d)) / (2 * a)
        root2 = (-b - math.sqrt(d)) / (2 * a)
        print("Roots are real and distinct:", root1, "and", root2)
    elif d == 0:
        root = -b / (2 * a)
        print("Roots are real and equal:", root, "and", root)
    else:
        root1 = (-b + cmath.sqrt(d)) / (2 * a)
        root2 = (-b - cmath.sqrt(d)) / (2 * a)
        print("Roots are complex:", root1, "and", root2)


for trial in range(2):
    print("Equation", trial + 1)
    a = float(input("Enter coefficient a: "))
    b = float(input("Enter coefficient b: "))
    c = float(input("Enter coefficient c: "))
    solve_quadratic(a, b, c)
```

Output:

```
Equation 1
Enter coefficient a: 1
Enter coefficient b: -5
Enter coefficient c: 6
Discriminant = 1.0
Roots are real and distinct: 3.0 and 2.0
Equation 2
Enter coefficient a: 1
Enter coefficient b: 2
Enter coefficient c: 5
Discriminant = -16.0
Roots are complex: (-1+2j) and (-1-2j)
```

Program 1.43: Distance between Two Points

Aim: To write a Python program that calculates the distance between two points (x1, y1) and (x2, y2).

Program:

```
import math

x1 = float(input("Enter x1: "))
y1 = float(input("Enter y1: "))
x2 = float(input("Enter x2: "))
y2 = float(input("Enter y2: "))

distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
print("Distance between the points =", distance)
print("Verified using math.dist():", math.dist((x1, y1), (x2, y2)))
```

Output:

```
Enter x1: 1
Enter y1: 2
Enter x2: 4
Enter y2: 6
Distance between the points = 5.0
Verified using math.dist(): 5.0
```

Program 1.44: Body Mass Index (BMI)

Aim: To write a Python program that calculates the Body Mass Index and displays the weight category.

Program:

```
weight = float(input("Enter weight in kg: "))
height_cm = float(input("Enter height in cm: "))

height_m = height_cm / 100
bmi = weight / height_m ** 2
print(f"BMI = {bmi:.2f}")
```

```
if bmi < 18.5:
    print("Category: Underweight")
elif 18.5 <= bmi < 25:
    print("Category: Normal weight")
elif 25 <= bmi < 30:
    print("Category: Overweight")
else:
    print("Category: Obese")
```

Output:

```
Enter weight in kg: 68
Enter height in cm: 172
BMI = 22.99
Category: Normal weight
```

Program 1.45: Simple vs Compound Interest (Yearly and Monthly)

Aim: To write a Python program that compares simple interest with compound interest compounded yearly and monthly for the same deposit.

Program:

```
principal = float(input("Enter principal (Rs): "))
rate = float(input("Enter annual rate (%): "))
years = int(input("Enter time (years): "))

simple_interest = principal * rate * years / 100
amount_yearly = principal * (1 + rate / 100) ** years
amount_monthly = principal * (1 + rate / (12 * 100)) ** (12 * years)

print(f"Simple interest           : Rs {simple_interest:.2f}")
print(f"Compound interest (yearly): Rs {amount_yearly - principal:.2f}")
print(f"Compound interest (monthly): Rs {amount_monthly - principal:.2f}")
print(f"Extra earned by monthly compounding over SI: Rs {amount_monthly - principal - sim
ple_interest:.2f}")
```

Output:

```
Enter principal (Rs): 100000
Enter annual rate (%): 8
Enter time (years): 5
Simple interest           : Rs 40000.00
Compound interest (yearly): Rs 46932.81
Compound interest (monthly): Rs 48984.57
Extra earned by monthly compounding over SI: Rs 8984.57
```

Program 1.46: Volume and Surface Area of Cylinder, Sphere and Cone

Aim: To write a Python program that calculates the volume and total surface area of a cylinder, a sphere and a cone.

Program:

```
import math

r = float(input("Enter radius: "))
```

```
h = float(input("Enter height (for cylinder and cone): "))

cyl_volume = math.pi * r ** 2 * h
cyl_area = 2 * math.pi * r * (r + h)

sph_volume = 4 / 3 * math.pi * r ** 3
sph_area = 4 * math.pi * r ** 2

slant = (r ** 2 + h ** 2) ** 0.5
cone_volume = math.pi * r ** 2 * h / 3
cone_area = math.pi * r * (r + slant)

print(f"Cylinder : volume = {cyl_volume:.2f}, surface area = {cyl_area:.2f}")
print(f"Sphere   : volume = {sph_volume:.2f}, surface area = {sph_area:.2f}")
print(f"Cone     : volume = {cone_volume:.2f}, surface area = {cone_area:.2f}")
```

Output:

```
Enter radius: 7
Enter height (for cylinder and cone): 10
Cylinder : volume = 1539.38, surface area = 747.70
Sphere   : volume = 1436.76, surface area = 615.75
Cone     : volume = 513.13, surface area = 422.37
```

Program 1.47: Sum of AP and GP Series using Formulas

Aim: To write a Python program that finds the nth term and the sum of n terms of an arithmetic progression and a geometric progression using formulas.

Program:

```
a = float(input("Enter first term a: "))
d = float(input("Enter common difference d (AP): "))
r = float(input("Enter common ratio r (GP, r != 1): "))
n = int(input("Enter number of terms n: "))

ap_nth = a + (n - 1) * d
ap_sum = n / 2 * (2 * a + (n - 1) * d)

gp_nth = a * r ** (n - 1)
gp_sum = a * (r ** n - 1) / (r - 1)

print(f"AP : {n}th term = {ap_nth}, sum of {n} terms = {ap_sum}")
print(f"GP : {n}th term = {gp_nth}, sum of {n} terms = {gp_sum}")
```

Output:

```
Enter first term a: 3
Enter common difference d (AP): 4
Enter common ratio r (GP, r != 1): 2
Enter number of terms n: 10
AP : 10th term = 39.0, sum of 10 terms = 210.0
GP : 10th term = 1536.0, sum of 10 terms = 3069.0
```

Program 1.48: Slope, Midpoint and Equation of a Line

Aim: To write a Python program that finds the slope, midpoint and y-intercept of the line joining two points.

Program:

```
x1 = float(input("Enter x1: "))
y1 = float(input("Enter y1: "))
x2 = float(input("Enter x2: "))
y2 = float(input("Enter y2: "))

slope = (y2 - y1) / (x2 - x1)
mid_x = (x1 + x2) / 2
mid_y = (y1 + y2) / 2
intercept = y1 - slope * x1

print(f"Slope       : {slope:.2f}")
print(f"Midpoint    : ({mid_x:.2f}, {mid_y:.2f})")
print(f"Equation    : y = {slope:.2f}x {intercept:+.2f}")
```

Output:

```
Enter x1: 2
Enter y1: 3
Enter x2: 6
Enter y2: 11
Slope       : 2.00
Midpoint    : (4.00, 7.00)
Equation    : y = 2.00x -1.00
```

Program 1.49: Equations of Motion

Aim: To write a Python program that applies the equations of motion to find the final velocity, distance travelled and the check value v^2 = u^2 + 2as.

Program:

```
u = float(input("Enter initial velocity u (m/s): "))
a = float(input("Enter acceleration a (m/s^2): "))
t = float(input("Enter time t (s): "))

v = u + a * t
s = u * t + 0.5 * a * t ** 2
v_squared = u ** 2 + 2 * a * s

print(f"Final velocity v        : {v:.2f} m/s")
print(f"Distance travelled s    : {s:.2f} m")
print(f"v^2 from v = u + at     : {v ** 2:.2f}")
print(f"v^2 from u^2 + 2as      : {v_squared:.2f}")
```

Output:

```
Enter initial velocity u (m/s): 5
Enter acceleration a (m/s^2): 2.5
Enter time t (s): 8
Final velocity v        : 25.00 m/s
```

```
Distance travelled s    : 120.00 m
v^2 from v = u + at     : 625.00
v^2 from u^2 + 2as      : 625.00
```

Program 1.50: Evaluate a Polynomial at a Given x

Aim: To write a Python program that evaluates the polynomial 3x^3 - 5x^2 + 2x - 7 at a given x, directly and by Horner's method.

Program:

```
x = float(input("Enter value of x: "))

direct = 3 * x ** 3 - 5 * x ** 2 + 2 * x - 7
horner = ((3 * x - 5) * x + 2) * x - 7

print(f"P(x) = 3x^3 - 5x^2 + 2x - 7 at x = {x}")
print(f"Direct evaluation : {direct}")
print(f"Horner's method   : {horner}")
```

Output:

```
Enter value of x: 2.5
P(x) = 3x^3 - 5x^2 + 2x - 7 at x = 2.5
Direct evaluation : 13.625
Horner's method   : 13.625
```

Program 1.51: Hypotenuse using Pythagoras Theorem

Aim: To write a Python program that finds the hypotenuse, area and perimeter of a right-angled triangle from its two shorter sides.

Program:

```
import math

a = float(input("Enter base: "))
b = float(input("Enter height: "))

c = (a ** 2 + b ** 2) ** 0.5
area = a * b / 2
perimeter = a + b + c

print(f"Hypotenuse  : {c:.2f}")
print(f"Using hypot : {math.hypot(a, b):.2f}")
print(f"Area        : {area:.2f}")
print(f"Perimeter   : {perimeter:.2f}")
```

Output:

```
Enter base: 9
Enter height: 12
Hypotenuse  : 15.00
Using hypot : 15.00
Area        : 54.00
Perimeter   : 36.00
```

Program 1.52: Percentage Increase or Decrease

Aim: To write a Python program that calculates the percentage change between an old value and a new value, such as a salary or price revision.

Program:

```
old_value = float(input("Enter old value: "))
new_value = float(input("Enter new value: "))

change = new_value - old_value
percent_change = (new_value - old_value) / old_value * 100
label = "increase" if change >= 0 else "decrease"

print(f"Change            : {change:.2f}")
print(f"Percentage {label}: {abs(percent_change):.2f}%")
# Without brackets the precedence gives a wrong answer
print(f"Wrong (no brackets): {new_value - old_value / old_value * 100:.2f}")
```

Output:

```
Enter old value: 42000
Enter new value: 46620
Change            : 4620.00
Percentage increase: 11.00%
Wrong (no brackets): 46520.00
```

Program 1.53: Kinetic and Potential Energy

Aim: To write a Python program that calculates the kinetic energy, potential energy and total mechanical energy of a moving body.

Program:

```
G = 9.8  # acceleration due to gravity (m/s^2)

mass = float(input("Enter mass (kg): "))
velocity = float(input("Enter velocity (m/s): "))
height = float(input("Enter height above ground (m): "))

kinetic = 0.5 * mass * velocity ** 2
potential = mass * G * height
total = kinetic + potential

print(f"Kinetic energy   : {kinetic:.2f} J")
print(f"Potential energy : {potential:.2f} J")
print(f"Total energy     : {total:.2f} J")
```

Output:

```
Enter mass (kg): 60
Enter velocity (m/s): 12
Enter height above ground (m): 15
Kinetic energy   : 4320.00 J
Potential energy : 8820.00 J
Total energy     : 13140.00 J
```

Problem Solving (Algorithms)

Program 1.54: Factorial of a Number

Aim: To write a Python program that finds the factorial of a given number.

Program:

```
n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    fact = 1
    i = 1
    while i <= n:
        fact = fact * i
        i = i + 1
    print("Factorial of", n, "is", fact)
```

Output:

```
Enter a number: 6
Factorial of 6 is 720
```

Program 1.55: Sum of Digits of a Number

Aim: To write a Python program that finds the sum of the digits of a given number.

Program:

```
num = int(input("Enter a number: "))
original = num
total = 0

while num > 0:
    digit = num % 10      # extract the last digit
    total = total + digit
    num = num // 10       # remove the last digit

print("Sum of digits of", original, "is", total)
```

Output:

```
Enter a number: 45873
Sum of digits of 45873 is 27
```

Program 1.56: Reverse a Number and Check Palindrome

Aim: To write a Python program that reverses a given number and checks whether it is a palindrome.

Program:

```
num = int(input("Enter a number: "))
original = num
rev = 0
```

```
while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num = num // 10

print("Reverse of", original, "is", rev)

if rev == original:
    print(original, "is a Palindrome")
else:
    print(original, "is not a Palindrome")
```

Output:

```
Enter a number: 12321
Reverse of 12321 is 12321
12321 is a Palindrome
```

Program 1.57: Check whether a Number is Prime

Aim: To write a Python program that checks whether a given number is prime.

Program:

```
n = int(input("Enter a number: "))

if n < 2:
    is_prime = False
else:
    is_prime = True
    i = 2
    # It is enough to check divisors up to the square root of n
    while i * i <= n:
        if n % i == 0:
            is_prime = False
            break
        i = i + 1

if is_prime:
    print(n, "is a Prime number")
else:
    print(n, "is not a Prime number")
```

Output:

```
Enter a number: 97
97 is a Prime number
```

Program 1.58: GCD and LCM using Euclid's Algorithm

Aim: To write a Python program that finds the GCD of two numbers using Euclid's algorithm and also finds their LCM.

Program:

```
import math

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

x, y = a, b
while y != 0:
    remainder = x % y
    x = y
    y = remainder

gcd = x
lcm = (a * b) // gcd
print("GCD of", a, "and", b, "is", gcd)
print("LCM of", a, "and", b, "is", lcm)
print("Verified using math.gcd():", math.gcd(a, b))
```

Output:

```
Enter first number: 48
Enter second number: 36
GCD of 48 and 36 is 12
LCM of 48 and 36 is 144
Verified using math.gcd(): 12
```

Program 1.59: Fibonacci Series

Aim: To write a Python program that prints the first n terms of the Fibonacci series.

Program:

```
n = int(input("Enter the number of terms: "))

if n <= 0:
    print("Please enter a positive integer")
else:
    a, b = 0, 1
    count = 0
    print("Fibonacci series:")
    while count < n:
        print(a, end=" ")
        a, b = b, a + b
        count = count + 1
    print()
```

Output:

```
Enter the number of terms: 10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
```

Unit II: Control Structures and Strings

Conditional Statements

Program 2.1: Positive, Negative or Zero

Aim: To write a Python program that checks whether a given number is positive, negative or zero.

Program:

```
# Check whether a number is positive, negative or zero
number = float(input("Enter a number: "))

if number > 0:
    print(number, "is a positive number")
elif number < 0:
    print(number, "is a negative number")
else:
    print("The number is zero")
```

Output:

```
Enter a number: -7.5
-7.5 is a negative number
```

Program 2.2: Even or Odd

Aim: To write a Python program that checks whether a given integer is even or odd.

Program:

```
# Check whether a number is even or odd
number = int(input("Enter an integer: "))

if number % 2 == 0:
    print(number, "is Even")
else:
    print(number, "is Odd")
```

Output:

```
Enter an integer: 37
37 is Odd
```

Program 2.3: Largest of Three Numbers (Nested if)

Aim: To write a Python program that finds the largest of three numbers using nested if statements.

Program:

```
# Largest of three numbers using nested if
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b:
```

```
    if a >= c:
        largest = a
    else:
        largest = c
else:
    if b >= c:
        largest = b
    else:
        largest = c

print("Largest number is:", largest)
```

Output:

```
Enter first number: 12
Enter second number: 45
Enter third number: 27
Largest number is: 45.0
```

Program 2.4: Grade Calculator (elif Ladder)

Aim: To write a Python program that assigns a grade to a student based on marks using an elif ladder.

Program:

```
# Grade calculator using if-elif ladder
marks = float(input("Enter marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks")
elif marks >= 90:
    print("Grade: O (Outstanding)")
elif marks >= 80:
    print("Grade: A+ (Excellent)")
elif marks >= 70:
    print("Grade: A (Very Good)")
elif marks >= 60:
    print("Grade: B+ (Good)")
elif marks >= 50:
    print("Grade: B (Average)")
elif marks >= 40:
    print("Grade: C (Pass)")
else:
    print("Grade: F (Fail)")
```

Output:

```
Enter marks (0-100): 76
Grade: A (Very Good)
```

Program 2.5: Leap Year Check

Aim: To write a Python program that checks whether a given year is a leap year.

Program:

```python
# Check whether a year is a leap year
year = int(input("Enter a year: "))

if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print(year, "is a Leap Year")
else:
    print(year, "is not a Leap Year")
```

Output:

```
Enter a year: 1900
1900 is not a Leap Year
```

Program 2.6: Vowel or Consonant

Aim: To write a Python program that checks whether an entered alphabet is a vowel or a consonant.

Program:

```python
# Check whether a character is a vowel or consonant
ch = input("Enter a single alphabet: ")

if len(ch) != 1 or not ch.isalpha():
    print("Invalid input. Please enter one alphabet.")
elif ch.lower() in "aeiou":
    print(ch, "is a Vowel")
else:
    print(ch, "is a Consonant")
```

Output:

```
Enter a single alphabet: E
E is a Vowel
```

Program 2.7: Simple Calculator using if-elif

Aim: To write a Python program that performs basic arithmetic operations based on the operator chosen by the user.

Program:

```python
# Simple calculator using if-elif
num1 = float(input("Enter first number: "))
op = input("Enter operator (+, -, *, /, %, **): ")
num2 = float(input("Enter second number: "))

if op == "+":
    print("Result:", num1 + num2)
elif op == "-":
    print("Result:", num1 - num2)
elif op == "*":
    print("Result:", num1 * num2)
elif op == "/":
    if num2 == 0:
        print("Error: Division by zero")
    else:
```

```
        print("Result:", num1 / num2)
elif op == "%":
    if num2 == 0:
        print("Error: Division by zero")
    else:
        print("Result:", num1 % num2)
elif op == "**":
    print("Result:", num1 ** num2)
else:
    print("Invalid operator")
```

Output:

```
Enter first number: 15
Enter operator (+, -, *, /, %, **): /
Enter second number: 4
Result: 3.75
```

Program 2.8: Triangle Validity and Type

Aim: To write a Python program that checks whether three sides form a valid triangle and finds its type.

Program:

```
# Check triangle validity and type
a = float(input("Enter side a: "))
b = float(input("Enter side b: "))
c = float(input("Enter side c: "))

if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("Valid triangle: Equilateral")
    elif a == b or b == c or a == c:
        print("Valid triangle: Isosceles")
    else:
        print("Valid triangle: Scalene")

    # Check for right-angled triangle (sorted sides)
    x, y, z = sorted([a, b, c])
    if abs(x * x + y * y - z * z) < 1e-9:
        print("It is also a Right-angled triangle")
else:
    print("These sides do not form a valid triangle")
```

Output:

```
Enter side a: 3
Enter side b: 4
Enter side c: 5
Valid triangle: Scalene
It is also a Right-angled triangle
```

Program 2.9: Electricity Bill (Slab Rates)

Aim: To write a Python program that calculates an electricity bill using slab-wise unit rates.

Program:

```python
# Electricity bill calculation using slab rates
units = int(input("Enter units consumed: "))

if units <= 100:
    amount = units * 1.50
elif units <= 200:
    amount = 100 * 1.50 + (units - 100) * 2.50
elif units <= 300:
    amount = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
else:
    amount = 100 * 1.50 + 100 * 2.50 + 100 * 4.00 + (units - 300) * 6.00

fixed_charge = 50
total = amount + fixed_charge

print("Units consumed  :", units)
print("Energy charges  : Rs", format(amount, ".2f"))
print("Fixed charges   : Rs", format(fixed_charge, ".2f"))
print("Total bill      : Rs", format(total, ".2f"))
```

Output:

```
Enter units consumed: 350
Units consumed  : 350
Energy charges  : Rs 1100.00
Fixed charges   : Rs 50.00
Total bill      : Rs 1150.00
```

Program 2.10: Character Type Check

Aim: To write a Python program that checks whether a character is an uppercase letter, lowercase letter, digit or special character.

Program:

```python
# Identify the type of a character
ch = input("Enter a character: ")

if len(ch) != 1:
    print("Please enter exactly one character")
elif ch.isupper():
    print(ch, "is an Uppercase letter")
elif ch.islower():
    print(ch, "is a Lowercase letter")
elif ch.isdigit():
    print(ch, "is a Digit")
else:
    print(ch, "is a Special character")
```

Output:

```
Enter a character: @
@ is a Special character
```

Program 2.11: Quadrant of a Point

Aim: To write a Python program that finds the quadrant or axis on which a point (x, y) lies.

Program:

```
# Find the quadrant of a point (x, y)
x = float(input("Enter x coordinate: "))
y = float(input("Enter y coordinate: "))

if x == 0 and y == 0:
    print("The point lies at the origin")
elif x == 0:
    print("The point lies on the Y-axis")
elif y == 0:
    print("The point lies on the X-axis")
elif x > 0 and y > 0:
    print("The point lies in Quadrant I")
elif x < 0 and y > 0:
    print("The point lies in Quadrant II")
elif x < 0 and y < 0:
    print("The point lies in Quadrant III")
else:
    print("The point lies in Quadrant IV")
```

Output:

```
Enter x coordinate: -3
Enter y coordinate: 4.5
The point lies in Quadrant II
```

Program 2.12: BMI Calculator with Category

Aim: To write a Python program that calculates Body Mass Index and displays the weight category.

Program:

```
# BMI calculator with weight category
weight = float(input("Enter weight in kg: "))
height_cm = float(input("Enter height in cm: "))

if weight <= 0 or height_cm <= 0:
    print("Weight and height must be positive")
else:
    height_m = height_cm / 100
    bmi = weight / (height_m ** 2)
    if bmi < 18.5:
        category = "Underweight"
    elif bmi < 25:
        category = "Normal weight"
    elif bmi < 30:
        category = "Overweight"
    else:
        category = "Obese"
    print(f"BMI = {bmi:.2f}")
    print("Category:", category)
```

Output:

```
Enter weight in kg: 72
Enter height in cm: 168
```

```
BMI = 25.51
Category: Overweight
```

Program 2.13: Triangle Classification by Angles

Aim: To write a Python program that checks whether three angles form a triangle and classifies it as acute, right or obtuse.

Program:

```
# Classify a triangle using its three angles
a = int(input("Enter first angle: "))
b = int(input("Enter second angle: "))
c = int(input("Enter third angle: "))

if a > 0 and b > 0 and c > 0 and a + b + c == 180:
    if a == 90 or b == 90 or c == 90:
        kind = "Right-angled triangle"
    elif a > 90 or b > 90 or c > 90:
        kind = "Obtuse-angled triangle"
    else:
        kind = "Acute-angled triangle"
    print("Valid triangle:", kind)
    if a == b == c:
        print("It is also equiangular (equilateral).")
else:
    print("These angles cannot form a triangle")
```

Output:

```
Enter first angle: 35
Enter second angle: 45
Enter third angle: 100
Valid triangle: Obtuse-angled triangle
```

Program 2.14: Cinema Ticket Price by Age and Day Type

Aim: To write a Python program that computes a cinema ticket price based on the viewer's age group and whether the day is a weekday or weekend.

Program:

```
# Ticket price based on age group and day type
age = int(input("Enter age: "))
day_type = input("Enter day type (weekday/weekend): ").strip().lower()

if age < 0:
    print("Invalid age")
else:
    if age < 12:
        group, price = "Child", 100
    elif age < 60:
        group, price = "Adult", 180
    else:
        group, price = "Senior citizen", 120
```

```
    if day_type == "weekend":
        price += 50
    elif day_type != "weekday":
        print("Unknown day type, weekday price applied")

    print("Age group  :", group)
    print("Ticket price: Rs.", price)
```

Output:

```
Enter age: 65
Enter day type (weekday/weekend): weekend
Age group  : Senior citizen
Ticket price: Rs. 170
```

Program 2.15: Discount by Purchase Amount Slabs

Aim: To write a Python program that applies a discount to a purchase based on amount slabs and prints the net bill.

Program:

```
# Discount on purchase amount using slabs
amount = float(input("Enter purchase amount (Rs.): "))

if amount < 0:
    print("Invalid amount")
else:
    if amount >= 10000:
        rate = 15
    elif amount >= 5000:
        rate = 10
    elif amount >= 1000:
        rate = 5
    else:
        rate = 0
    discount = amount * rate / 100
    net = amount - discount
    print(f"Purchase amount : Rs. {amount:.2f}")
    print(f"Discount ({rate}%)  : Rs. {discount:.2f}")
    print(f"Amount to pay   : Rs. {net:.2f}")
```

Output:

```
Enter purchase amount (Rs.): 7450
Purchase amount : Rs. 7450.00
Discount (10%)  : Rs. 745.00
Amount to pay   : Rs. 6705.00
```

Program 2.16: Shipping Charge by Weight and Zone

Aim: To write a Python program that calculates the courier shipping charge from parcel weight and delivery zone.

Program:

```
# Courier shipping charge based on weight and zone
weight = float(input("Enter parcel weight in kg: "))
zone = input("Enter zone (local/state/national): ").strip().lower()

if weight <= 0 or weight > 30:
    print("Weight must be between 0 and 30 kg")
elif zone not in ("local", "state", "national"):
    print("Invalid zone")
else:
    if zone == "local":
        base, per_kg = 40, 20
    elif zone == "state":
        base, per_kg = 60, 30
    else:
        base, per_kg = 90, 45

    extra_kg = 0
    if weight > 1:
        extra_kg = int(weight - 1)
        if weight - 1 > extra_kg:      # round up part kilogram
            extra_kg += 1
    charge = base + extra_kg * per_kg
    print("Zone            :", zone.title())
    print("Extra kg charged:", extra_kg)
    print("Shipping charge : Rs.", charge)
```

Output:

```
Enter parcel weight in kg: 3.4
Enter zone (local/state/national): state
Zone            : State
Extra kg charged: 3
Shipping charge : Rs. 150
```

Program 2.17: ATM Withdrawal Validation

Aim: To write a Python program that validates an ATM withdrawal request against balance, minimum balance, and multiples of 100.

Program:

```
# Validate an ATM withdrawal request
balance = float(input("Enter account balance: "))
amount = int(input("Enter amount to withdraw: "))

MIN_BALANCE = 1000
LIMIT = 20000

if amount <= 0:
    print("Amount must be positive")
elif amount % 100 != 0:
    print("Amount must be in multiples of 100")
elif amount > LIMIT:
    print("Exceeds per-transaction limit of Rs.", LIMIT)
elif balance - amount < MIN_BALANCE:
    print("Insufficient funds. Minimum balance of Rs.", MIN_BALANCE, "must be kept")
else:
```

```
    balance -= amount
    print("Please collect your cash: Rs.", amount)
    print(f"Available balance: Rs. {balance:.2f}")
```

Output:

```
Enter account balance: 15250
Enter amount to withdraw: 4500
Please collect your cash: Rs. 4500
Available balance: Rs. 10750.00
```

Program 2.18: Admission Eligibility Check

Aim: To write a Python program that checks engineering admission eligibility using subject-wise and aggregate mark rules.

Program:

```
# Admission eligibility based on marks
maths = int(input("Enter Maths marks: "))
physics = int(input("Enter Physics marks: "))
chemistry = int(input("Enter Chemistry marks: "))

total = maths + physics + chemistry
rule1 = maths >= 60 and physics >= 50 and chemistry >= 40 and total >= 200
rule2 = maths + physics >= 150

print("Total marks:", total)
if rule1 or rule2:
    print("Eligible for admission")
    if rule1:
        print("Reason: meets all subject and aggregate requirements")
    else:
        print("Reason: Maths + Physics is at least 150")
else:
    print("Not eligible for admission")
```

Output:

```
Enter Maths marks: 78
Enter Physics marks: 74
Enter Chemistry marks: 35
Total marks: 187
Eligible for admission
Reason: Maths + Physics is at least 150
```

Program 2.19: Income Tax Calculation by Slabs

Aim: To write a Python program that computes income tax on annual taxable income using progressive slab rates.

Program:

```
# Income tax using progressive slabs (illustrative rates)
income = float(input("Enter annual taxable income (Rs.): "))

if income <= 300000:
```

```
    tax = 0
elif income <= 700000:
    tax = (income - 300000) * 0.05
elif income <= 1000000:
    tax = 20000 + (income - 700000) * 0.10
elif income <= 1200000:
    tax = 50000 + (income - 1000000) * 0.15
elif income <= 1500000:
    tax = 80000 + (income - 1200000) * 0.20
else:
    tax = 140000 + (income - 1500000) * 0.30

cess = tax * 0.04
print(f"Income tax : Rs. {tax:.2f}")
print(f"Cess (4%)  : Rs. {cess:.2f}")
print(f"Total tax  : Rs. {tax + cess:.2f}")
```

Output:

```
Enter annual taxable income (Rs.): 1350000
Income tax : Rs. 110000.00
Cess (4%)  : Rs. 4400.00
Total tax  : Rs. 114400.00
```

Program 2.20: Nature of Roots of a Quadratic Equation

Aim: To write a Python program that finds the nature of roots of ax^2 + bx + c = 0 and computes them.

Program:

```
# Nature and values of roots of a quadratic equation
a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))

if a == 0:
    print("Not a quadratic equation (a = 0)")
else:
    d = b * b - 4 * a * c
    print("Discriminant =", d)
    if d > 0:
        r1 = (-b + d ** 0.5) / (2 * a)
        r2 = (-b - d ** 0.5) / (2 * a)
        print("Roots are real and distinct")
        print(f"Root 1 = {r1:.3f}, Root 2 = {r2:.3f}")
    elif d == 0:
        r = -b / (2 * a)
        print("Roots are real and equal")
        print(f"Root = {r:.3f}")
    else:
        real = -b / (2 * a)
        imag = (-d) ** 0.5 / (2 * a)
        print("Roots are complex (imaginary)")
        print(f"Root 1 = {real:.3f} + {imag:.3f}i")
        print(f"Root 2 = {real:.3f} - {imag:.3f}i")
```

Output:

```
Enter a: 2
Enter b: 4
Enter c: 5
Discriminant = -24.0
Roots are complex (imaginary)
Root 1 = -1.000 + 1.225i
Root 2 = -1.000 - 1.225i
```

Iterative Statements

Program 2.21: Sum of First n Natural Numbers (for and while)

Aim: To write a Python program that finds the sum of the first n natural numbers using both for and while loops.

Program:

```python
# Sum of first n natural numbers using for and while loops
n = int(input("Enter n: "))

# Using for loop
total_for = 0
for i in range(1, n + 1):
    total_for += i

# Using while loop
total_while = 0
i = 1
while i <= n:
    total_while += i
    i += 1

print("Sum using for loop   :", total_for)
print("Sum using while loop :", total_while)
print("Sum using formula    :", n * (n + 1) // 2)
```

Output:

```
Enter n: 10
Sum using for loop   : 55
Sum using while loop : 55
Sum using formula    : 55
```

Program 2.22: Multiplication Table

Aim: To write a Python program that prints the multiplication table of a given number.

Program:

```python
# Multiplication table of a number
number = int(input("Enter a number: "))

for i in range(1, 11):
    print(f"{number} x {i:2} = {number * i}")
```

Output:

```
Enter a number: 7
7 x  1 = 7
7 x  2 = 14
7 x  3 = 21
7 x  4 = 28
7 x  5 = 35
7 x  6 = 42
7 x  7 = 49
7 x  8 = 56
7 x  9 = 63
7 x 10 = 70
```

Program 2.23: Factorial of a Number

Aim: To write a Python program that finds the factorial of a given number using a loop.

Program:

```python
# Factorial of a number using a for loop
n = int(input("Enter a non-negative integer: "))

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    print(f"Factorial of {n} is {fact}")
```

Output:

```
Enter a non-negative integer: 6
Factorial of 6 is 720
```

Program 2.24: Fibonacci Series

Aim: To write a Python program that prints the first n terms of the Fibonacci series.

Program:

```python
# Print first n terms of the Fibonacci series
n = int(input("Enter number of terms: "))

a, b = 0, 1
print("Fibonacci series:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Output:

```
Enter number of terms: 10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
```

Program 2.25: Prime Check and Primes in a Range

Aim: To write a Python program that checks whether a number is prime and prints all primes in a given range.

Program:

```
# Prime check and prime numbers in a range
num = int(input("Enter a number to check: "))

if num < 2:
    print(num, "is not a prime number")
else:
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a prime number")

low = int(input("Enter lower limit: "))
high = int(input("Enter upper limit: "))

print(f"Prime numbers between {low} and {high}:")
for n in range(max(low, 2), high + 1):
    is_prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            is_prime = False
            break
    if is_prime:
        print(n, end=" ")
print()
```

Output:

```
Enter a number to check: 29
29 is a prime number
Enter lower limit: 10
Enter upper limit: 50
Prime numbers between 10 and 50:
11 13 17 19 23 29 31 37 41 43 47
```

Program 2.26: Armstrong Number

Aim: To write a Python program that checks whether a given number is an Armstrong number.

Program:

```
# Check whether a number is an Armstrong number
num = int(input("Enter a number: "))

digits = len(str(num))
temp = num
total = 0
while temp > 0:
    digit = temp % 10
```

```
    total += digit ** digits
    temp //= 10

if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")
```

Output:

```
Enter a number: 153
153 is an Armstrong number
```

Program 2.27: Palindrome Number

Aim: To write a Python program that checks whether a given number is a palindrome.

Program:

```
# Check whether a number is a palindrome
num = int(input("Enter a number: "))

temp = num
reverse = 0
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp //= 10

if reverse == num:
    print(num, "is a Palindrome number")
else:
    print(num, "is not a Palindrome number")
```

Output:

```
Enter a number: 12321
12321 is a Palindrome number
```

Program 2.28: Perfect Number

Aim: To write a Python program that checks whether a given number is a perfect number.

Program:

```
# Check whether a number is a perfect number
num = int(input("Enter a number: "))

divisor_sum = 0
for i in range(1, num // 2 + 1):
    if num % i == 0:
        divisor_sum += i

if num > 0 and divisor_sum == num:
    print(num, "is a Perfect number")
else:
    print(num, "is not a Perfect number")
```

Output:

```
Enter a number: 28
28 is a Perfect number
```

Program 2.29: GCD and LCM of Two Numbers

Aim: To write a Python program that finds the GCD and LCM of two numbers using the Euclidean algorithm.

Program:

```
# GCD and LCM using the Euclidean algorithm
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

a, b = x, y
while b != 0:
    a, b = b, a % b
gcd = a

lcm = (x * y) // gcd
print("GCD of", x, "and", y, "is", gcd)
print("LCM of", x, "and", y, "is", lcm)
```

Output:

```
Enter first number: 36
Enter second number: 60
GCD of 36 and 60 is 12
LCM of 36 and 60 is 180
```

Program 2.30: Sum of Digits and Reverse of a Number

Aim: To write a Python program that finds the sum of digits and the reverse of a number using a while loop.

Program:

```
# Sum of digits and reverse of a number using while loop
num = int(input("Enter a positive integer: "))

temp = num
digit_sum = 0
reverse = 0
while temp > 0:
    digit = temp % 10
    digit_sum += digit
    reverse = reverse * 10 + digit
    temp //= 10

print("Sum of digits :", digit_sum)
print("Reversed number:", reverse)
```

Output:

```
Enter a positive integer: 45873
Sum of digits : 27
Reversed number: 37854
```

Program 2.31: Demonstrating break, continue and pass

Aim: To write a Python program that demonstrates the use of break, continue and pass statements.

Program:

```
# Demonstration of break, continue and pass
limit = int(input("Enter a limit: "))

print("break: stop when a multiple of 7 is found")
for i in range(1, limit + 1):
    if i % 7 == 0:
        print("Found", i, "- breaking out")
        break
    print(i, end=" ")
print()

print("continue: skip even numbers")
for i in range(1, limit + 1):
    if i % 2 == 0:
        continue
    print(i, end=" ")
print()

print("pass: placeholder for multiples of 3")
for i in range(1, limit + 1):
    if i % 3 == 0:
        pass  # to be implemented later
    else:
        print(i, end=" ")
print()
```

Output:

```
Enter a limit: 15
break: stop when a multiple of 7 is found
1 2 3 4 5 6 Found 7 - breaking out

continue: skip even numbers
1 3 5 7 9 11 13 15
pass: placeholder for multiples of 3
1 2 4 5 7 8 10 11 13 14
```

Program 2.32: Number Guessing Game

Aim: To write a Python program for a number guessing game with a fixed secret number and a limited number of attempts.

Program:

```
# Number guessing game with fixed secret and limited attempts
SECRET = 42
```

```
MAX_ATTEMPTS = 5

print("Guess the number between 1 and 100. You have", MAX_ATTEMPTS, "attempts.")
for attempt in range(1, MAX_ATTEMPTS + 1):
    guess = int(input(f"Attempt {attempt}: Enter your guess: "))
    if guess == SECRET:
        print("Correct! You guessed it in", attempt, "attempt(s).")
        break
    elif guess < SECRET:
        print("Too low!")
    else:
        print("Too high!")
else:
    print("Sorry, out of attempts. The number was", SECRET)
```

Output:

```
Guess the number between 1 and 100. You have 5 attempts.
Attempt 1: Enter your guess: 50
Too high!
Attempt 2: Enter your guess: 25
Too low!
Attempt 3: Enter your guess: 40
Too low!
Attempt 4: Enter your guess: 42
Correct! You guessed it in 4 attempt(s).
```

Program 2.33: Strong Number

Aim: To write a Python program that checks whether a number is a strong number.

Program:

```
# Check whether a number is a strong number
num = int(input("Enter a number: "))

temp = num
total = 0
while temp > 0:
    digit = temp % 10
    fact = 1
    for i in range(1, digit + 1):
        fact *= i
    total += fact
    temp //= 10

if num > 0 and total == num:
    print(num, "is a Strong number")
else:
    print(num, "is not a Strong number")
```

Output:

```
Enter a number: 145
145 is a Strong number
```

Program 2.34: Shopping Cart Total until 0 is Entered

Aim: To write a Python program that reads item prices and quantities until 0 is entered and prints the bill with GST.

Program:

```
# Shopping cart: read prices until 0 is entered
total = 0.0
items = 0

while True:
    price = float(input("Enter item price (0 to finish): "))
    if price == 0:
        break
    if price < 0:
        print("Invalid price, skipped")
        continue
    qty = int(input("Enter quantity: "))
    line_total = price * qty
    items += qty
    total += line_total
    print(f"  Added: {qty} x {price:.2f} = {line_total:.2f}")

gst = total * 0.05
print("Number of items:", items)
print(f"Subtotal : Rs. {total:.2f}")
print(f"GST (5%) : Rs. {gst:.2f}")
print(f"Total    : Rs. {total + gst:.2f}")
```

Output:

```
Enter item price (0 to finish): 120
Enter quantity: 2
  Added: 2 x 120.00 = 240.00
Enter item price (0 to finish): -5
Invalid price, skipped
Enter item price (0 to finish): 45.5
Enter quantity: 4
  Added: 4 x 45.50 = 182.00
Enter item price (0 to finish): 299
Enter quantity: 1
  Added: 1 x 299.00 = 299.00
Enter item price (0 to finish): 0
Number of items: 7
Subtotal : Rs. 721.00
GST (5%) : Rs. 36.05
Total    : Rs. 757.05
```

Program 2.35: PIN Verification with 3 Attempts

Aim: To write a Python program that allows a user at most three attempts to enter the correct PIN.

Program:

```
# PIN verification with a maximum of 3 attempts
CORRECT_PIN = "4826"
```

```
for attempt in range(1, 4):
    pin = input(f"Attempt {attempt}: Enter 4-digit PIN: ")
    if pin == CORRECT_PIN:
        print("PIN verified. Access granted.")
        break
    print("Wrong PIN.", 3 - attempt, "attempt(s) left")
else:
    print("Card blocked after 3 wrong attempts.")
```

Output:

```
Attempt 1: Enter 4-digit PIN: 1234
Wrong PIN. 2 attempt(s) left
Attempt 2: Enter 4-digit PIN: 4826
PIN verified. Access granted.
```

Program 2.36: ATM Menu with Balance (Loop Driven)

Aim: To write a Python program that simulates an ATM menu for deposit, withdrawal and balance enquiry until the user chooses exit.

Program:

```
# ATM simulation with a menu-driven loop
balance = 5000.0
choice = 0

while choice != 4:
    print("1.Deposit 2.Withdraw 3.Balance 4.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        amount = float(input("Enter amount to deposit: "))
        if amount > 0:
            balance += amount
            print(f"Deposited Rs. {amount:.2f}")
        else:
            print("Invalid amount")
    elif choice == 2:
        amount = int(input("Enter amount to withdraw: "))
        if amount <= 0 or amount % 100 != 0:
            print("Enter a positive multiple of 100")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            print("Collect cash: Rs.", amount)
    elif choice == 3:
        print(f"Current balance: Rs. {balance:.2f}")
    elif choice == 4:
        print("Thank you for banking with us.")
    else:
        print("Invalid choice")
```

Output:

```
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 3
```

```
Current balance: Rs. 5000.00
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 1
Enter amount to deposit: 2500
Deposited Rs. 2500.00
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 2
Enter amount to withdraw: 750
Enter a positive multiple of 100
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 2
Enter amount to withdraw: 3000
Collect cash: Rs. 3000
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 3
Current balance: Rs. 4500.00
1.Deposit 2.Withdraw 3.Balance 4.Exit
Enter choice: 4
Thank you for banking with us.
```

Program 2.37: Grade Sheet for n Students

Aim: To write a Python program that reads marks of three subjects for n students and prints total, average, grade and class average.

Program:

```python
# Grade sheet for n students
n = int(input("Enter number of students: "))
class_total = 0

print(f"{'Name':<10}{'Total':>6}{'Avg':>8}{'Grade':>7}")
for i in range(n):
    name = input(f"Student {i + 1} name: ")
    m1, m2, m3 = map(int, input("Marks in 3 subjects: ").split())
    total = m1 + m2 + m3
    avg = total / 3
    if min(m1, m2, m3) < 40:
        grade = "F"
    elif avg >= 90:
        grade = "O"
    elif avg >= 75:
        grade = "A"
    elif avg >= 60:
        grade = "B"
    else:
        grade = "C"
    class_total += avg
    print(f"{name:<10}{total:>6}{avg:>8.2f}{grade:>7}")

print(f"Class average: {class_total / n:.2f}")
```

Output:

```
Enter number of students: 3
Name       Total     Avg  Grade
Student 1 name: Anil
```

```
Marks in 3 subjects: 88 92 79
Anil         259   86.33      A
Student 2 name: Bhavya
Marks in 3 subjects: 95 91 97
Bhavya       283   94.33      O
Student 3 name: Charan
Marks in 3 subjects: 55 38 70
Charan       163   54.33      F
Class average: 78.33
```

Program 2.38: Sum of Series (Harmonic and Alternating Power)

Aim: To write a Python program that computes 1 + 1/2 + … + 1/n and 1 - x + x^2 - x^3 + … up to n terms.

Program:

```python
# Sum of two series using loops
n = int(input("Enter number of terms n: "))
x = float(input("Enter x: "))

harmonic = 0.0
for i in range(1, n + 1):
    harmonic += 1 / i
print(f"1 + 1/2 + ... + 1/{n} = {harmonic:.4f}")

series = 0.0
term = 1.0
for i in range(n):
    series += term
    term *= -x          # next term changes sign and gains one power of x
print(f"1 - x + x^2 - ... ({n} terms, x = {x}) = {series:.4f}")
```

Output:

```
Enter number of terms n: 6
Enter x: 0.5
1 + 1/2 + ... + 1/6 = 2.4500
1 - x + x^2 - ... (6 terms, x = 0.5) = 0.6562
```

Program 2.39: Binary to Decimal and Decimal to Binary

Aim: To write a Python program that converts binary to decimal and decimal to binary using loops (without bin/int base).

Program:

```python
# Number system conversion using loops
binary = input("Enter a binary number: ")
decimal_value = 0
valid = True
for digit in binary:
    if digit not in "01":
        valid = False
        break
    decimal_value = decimal_value * 2 + int(digit)
```

```
if valid:
    print("Decimal value of", binary, "is", decimal_value)
else:
    print("Invalid binary number")

number = int(input("Enter a decimal number: "))
if number == 0:
    result = "0"
else:
    result = ""
    temp = number
    while temp > 0:
        result = str(temp % 2) + result
        temp //= 2
print("Binary value of", number, "is", result)
```

Output:

```
Enter a binary number: 110101
Decimal value of 110101 is 53
Enter a decimal number: 45
Binary value of 45 is 101101
```

Program 2.40: Prime Factors and Sum of Primes

Aim: To write a Python program that prints the prime factors of a number and the sum of all primes up to n.

Program:

```
# Prime factorisation and sum of primes up to n
number = int(input("Enter a number to factorise: "))
temp = number
d = 2
factors = []
while d * d <= temp:
    while temp % d == 0:
        factors.append(d)
        temp //= d
    d += 1
if temp > 1:
    factors.append(temp)
print("Prime factors of", number, ":", " x ".join(str(f) for f in factors))

n = int(input("Enter limit n: "))
total = 0
count = 0
for num in range(2, n + 1):
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        total += num
        count += 1
```

```
print("Number of primes up to", n, ":", count)
print("Sum of primes up to", n, ":", total)
```

Output:

```
Enter a number to factorise: 360
Prime factors of 360 : 2 x 2 x 2 x 3 x 3 x 5
Enter limit n: 50
Number of primes up to 50 : 15
Sum of primes up to 50 : 328
```

Program 2.41: Collatz Sequence

Aim: To write a Python program that prints the Collatz (3n + 1) sequence for a number and counts its steps.

Program:

```
# Collatz (3n + 1) sequence
n = int(input("Enter a positive integer: "))

if n <= 0:
    print("Enter a positive integer")
else:
    steps = 0
    peak = n
    print(n, end="")
    while n != 1:
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
        steps += 1
        peak = max(peak, n)
        print(" ->", n, end="")
    print()
    print("Steps taken:", steps)
    print("Highest value:", peak)
```

Output:

```
Enter a positive integer: 27
27 -> 82 -> 41 -> 124 -> 62 -> 31 -> 94 -> 47 -> 142 -> 71 -> 214 -> 107 -> 322 -> 161 ->
484 -> 242 -> 121 -> 364 -> 182 -> 91 -> 274 -> 137 -> 412 -> 206 -> 103 -> 310 -> 155 ->
466 -> 233 -> 700 -> 350 -> 175 -> 526 -> 263 -> 790 -> 395 -> 1186 -> 593 -> 1780 -> 890
-> 445 -> 1336 -> 668 -> 334 -> 167 -> 502 -> 251 -> 754 -> 377 -> 1132 -> 566 -> 283 ->
850 -> 425 -> 1276 -> 638 -> 319 -> 958 -> 479 -> 1438 -> 719 -> 2158 -> 1079 -> 3238 ->
1619 -> 4858 -> 2429 -> 7288 -> 3644 -> 1822 -> 911 -> 2734 -> 1367 -> 4102 -> 2051 -> 61
54 -> 3077 -> 9232 -> 4616 -> 2308 -> 1154 -> 577 -> 1732 -> 866 -> 433 -> 1300 -> 650 ->
325 -> 976 -> 488 -> 244 -> 122 -> 61 -> 184 -> 92 -> 46 -> 23 -> 70 -> 35 -> 106 -> 53 -
> 160 -> 80 -> 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
Steps taken: 111
Highest value: 9232
```

Program 2.42: Number to Words (0-999)

Aim: To write a Python program that converts a number from 0 to 999 into English words, e.g. for cheque writing.

Program:

```
# Convert numbers 0-999 to words
ONES = ["zero", "one", "two", "three", "four", "five", "six", "seven",
        "eight", "nine", "ten", "eleven", "twelve", "thirteen", "fourteen",
        "fifteen", "sixteen", "seventeen", "eighteen", "nineteen"]
TENS = ["", "", "twenty", "thirty", "forty", "fifty", "sixty", "seventy",
        "eighty", "ninety"]

while True:
    num = int(input("Enter a number 0-999 (-1 to stop): "))
    if num == -1:
        break
    if num < 0 or num > 999:
        print("Out of range")
        continue
    if num < 20:
        words = ONES[num]
    else:
        words = ""
        hundreds, rest = num // 100, num % 100
        if hundreds > 0:
            words = ONES[hundreds] + " hundred"
            if rest > 0:
                words += " and "
        if rest >= 20:
            words += TENS[rest // 10]
            if rest % 10 > 0:
                words += "-" + ONES[rest % 10]
        elif rest > 0:
            words += ONES[rest]
    print(num, "=", words.capitalize())
```

Output:

```
Enter a number 0-999 (-1 to stop): 7
7 = Seven
Enter a number 0-999 (-1 to stop): 45
45 = Forty-five
Enter a number 0-999 (-1 to stop): 300
300 = Three hundred
Enter a number 0-999 (-1 to stop): 918
918 = Nine hundred and eighteen
Enter a number 0-999 (-1 to stop): -1
```

Program 2.43: Decimal to Roman Numeral Conversion

Aim: To write a Python program that converts an integer from 1 to 3999 into a Roman numeral.

Program:

```
# Convert decimal number to Roman numeral
values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]
```

```
symbols = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"]

number = int(input("Enter a number (1-3999): "))

if number < 1 or number > 3999:
    print("Out of range")
else:
    temp = number
    roman = ""
    for i in range(len(values)):
        while temp >= values[i]:
            roman += symbols[i]
            temp -= values[i]
    print(number, "in Roman numerals is", roman)
```

Output:

```
Enter a number (1-3999): 2026
2026 in Roman numerals is MMXXVI
```

Program 2.44: Compound Interest Year-by-Year Table

Aim: To write a Python program that prints a year-wise table of opening balance, interest and closing balance for a fixed deposit.

Program:

```
# Year-by-year compound interest table
principal = float(input("Enter principal amount: "))
rate = float(input("Enter annual interest rate (%): "))
years = int(input("Enter number of years: "))

balance = principal
print(f"{'Year':<6}{'Opening':>12}{'Interest':>12}{'Closing':>12}")
for year in range(1, years + 1):
    interest = balance * rate / 100
    closing = balance + interest
    print(f"{year:<6}{balance:>12.2f}{interest:>12.2f}{closing:>12.2f}")
    balance = closing

print(f"Total interest earned: {balance - principal:.2f}")
```

Output:

```
Enter principal amount: 100000
Enter annual interest rate (%): 7.5
Enter number of years: 5
Year       Opening    Interest     Closing
1        100000.00     7500.00   107500.00
2        107500.00     8062.50   115562.50
3        115562.50     8667.19   124229.69
4        124229.69     9317.23   133546.91
5        133546.91    10016.02   143562.93
Total interest earned: 43562.93
```

Program 2.45: Loan EMI and Amortisation (First 6 Months)

Aim: To write a Python program that calculates the monthly EMI of a loan and prints the amortisation schedule for the first six months.

Program:

```
# Loan EMI and amortisation schedule for first 6 months
principal = float(input("Enter loan amount: "))
annual_rate = float(input("Enter annual interest rate (%): "))
months = int(input("Enter tenure in months: "))

r = annual_rate / (12 * 100)
emi = principal * r * (1 + r) ** months / ((1 + r) ** months - 1)
print(f"Monthly EMI: {emi:.2f}")

balance = principal
print(f"{'Month':<7}{'EMI':>10}{'Interest':>10}{'Principal':>11}{'Balance':>12}")
for month in range(1, min(6, months) + 1):
    interest = balance * r
    principal_part = emi - interest
    balance -= principal_part
    print(f"{month:<7}{emi:>10.2f}{interest:>10.2f}{principal_part:>11.2f}{balance:>12.2f
}")

print(f"Total payable over tenure: {emi * months:.2f}")
```

Output:

```
Enter loan amount: 500000
Enter annual interest rate (%): 9
Enter tenure in months: 60
Monthly EMI: 10379.18
Month         EMI  Interest  Principal     Balance
1        10379.18   3750.00    6629.18   493370.82
2        10379.18   3700.28    6678.90   486691.93
3        10379.18   3650.19    6728.99   479962.94
4        10379.18   3599.72    6779.46   473183.48
5        10379.18   3548.88    6830.30   466353.18
6        10379.18   3497.65    6881.53   459471.65
Total payable over tenure: 622750.66
```

Pattern Generation Programs

Program 2.46: Right-Angled Triangle of Stars

Aim: To write a Python program that prints a right-angled triangle star pattern.

Program:

```
# Right-angled triangle star pattern
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    print("* " * i)
```

Output:

```
Enter number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

Program 2.47: Inverted Right Triangle

Aim: To write a Python program that prints an inverted right-angled triangle star pattern.

Program:

```
# Inverted right-angled triangle using nested loops
rows = int(input("Enter number of rows: "))

for i in range(rows, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output:

```
Enter number of rows: 5
* * * * *
* * * *
* * *
* *
*
```

Program 2.48: Pyramid Pattern

Aim: To write a Python program that prints a centred pyramid of stars.

Program:

```
# Centred pyramid star pattern
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    spaces = " " * (rows - i)
    stars = "*" * (2 * i - 1)
    print(spaces + stars)
```

Output:

```
Enter number of rows: 5
    *
   ***
  *****
 *******
*********
```

Program 2.49: Diamond Pattern

Aim: To write a Python program that prints a diamond pattern of stars.

Program:

```
# Diamond star pattern
n = int(input("Enter number of rows for the upper half: "))

# Upper half
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))

# Lower half
for i in range(n - 1, 0, -1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

Output:

```
Enter number of rows for the upper half: 4
   *
  ***
 *****
*******
 *****
  ***
   *
```

Program 2.50: Number Triangle

Aim: To write a Python program that prints a number triangle where row i contains numbers 1 to i.

Program:

```
# Number triangle pattern
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output:

```
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Program 2.51: Floyd's Triangle

Aim: To write a Python program that prints Floyd's triangle.

Program:

```
# Floyd's triangle
rows = int(input("Enter number of rows: "))
```

```
number = 1
for i in range(1, rows + 1):
    for j in range(i):
        print(f"{number:3}", end=" ")
        number += 1
    print()
```

Output:

```
Enter number of rows: 5
  1
  2   3
  4   5   6
  7   8   9  10
 11  12  13  14  15
```

Program 2.52: Pascal's Triangle

Aim: To write a Python program that prints Pascal's triangle.

Program:

```
# Pascal's triangle using binomial coefficients
rows = int(input("Enter number of rows: "))

for i in range(rows):
    print(" " * (rows - i - 1) * 2, end="")
    value = 1
    for j in range(i + 1):
        print(f"{value:4}", end="")
        value = value * (i - j) // (j + 1)
    print()
```

Output:

```
Enter number of rows: 6
              1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10   5   1
```

Program 2.53: Alphabet Triangle Pattern

Aim: To write a Python program that prints a triangle pattern of alphabets.

Program:

```
# Alphabet triangle pattern
rows = int(input("Enter number of rows (max 26): "))

for i in range(1, rows + 1):
    for j in range(i):
        print(chr(65 + j), end=" ")
    print()
```

Output:

```
Enter number of rows (max 26): 5
A
A B
A B C
A B C D
A B C D E
```

Program 2.54: Hollow Square Pattern

Aim: To write a Python program that prints a hollow square of stars.

Program:

```
# Hollow square star pattern
n = int(input("Enter size of the square: "))

for i in range(n):
    for j in range(n):
        if i == 0 or i == n - 1 or j == 0 or j == n - 1:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

Output:

```
Enter size of the square: 5
* * * * *
*       *
*       *
*       *
* * * * *
```

Program 2.55: Number Pyramid

Aim: To write a Python program that prints a centred pyramid of numbers such as 1, 1 2 1, 1 2 3 2 1.

Program:

```
# Number pyramid (palindromic rows)
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    print("  " * (n - i), end="")
    for j in range(1, i + 1):
        print(j, end=" ")
    for j in range(i - 1, 0, -1):
        print(j, end=" ")
    print()
```

Output:

```
Enter number of rows: 5
        1
      1 2 1
    1 2 3 2 1
```

```
  1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

Program 2.56: Inverted Number Pyramid

Aim: To write a Python program that prints an inverted centred pyramid of numbers.

Program:

```
# Inverted number pyramid
n = int(input("Enter number of rows: "))

for i in range(n, 0, -1):
    print(" " * (n - i), end="")
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output:

```
Enter number of rows: 5
1 2 3 4 5
 1 2 3 4
  1 2 3
   1 2
    1
```

Program 2.57: Hollow Pyramid of Stars

Aim: To write a Python program that prints a hollow pyramid of stars.

Program:

```
# Hollow pyramid pattern
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(1, 2 * i):
        if j == 1 or j == 2 * i - 1 or i == n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output:

```
Enter number of rows: 6
     *
    * *
   *   *
  *     *
 *       *
***********
```

Program 2.58: Hourglass Pattern of Stars

Aim: To write a Python program that prints an hourglass (sand-timer) pattern of stars.

Program:

```
# Hourglass pattern
n = int(input("Enter number of rows for each half: "))

for i in range(n, 0, -1):          # upper half
    print(" " * (n - i) + "* " * i)
for i in range(2, n + 1):          # lower half
    print(" " * (n - i) + "* " * i)
```

Output:

```
Enter number of rows for each half: 5
* * * * *
 * * * *
  * * *
   * *
    *
   * *
  * * *
 * * * *
* * * * *
```

Program 2.59: Star X Pattern

Aim: To write a Python program that prints the letter X using stars in an n x n grid.

Program:

```
# X pattern using stars
n = int(input("Enter size (odd number): "))

for i in range(n):
    for j in range(n):
        if i == j or i + j == n - 1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output:

```
Enter size (odd number): 7
*     *
 *   *
  * *
   *
  * *
 *   *
*     *
```

Program 2.60: Butterfly Pattern

Aim: To write a Python program that prints a butterfly pattern of stars.

Program:

```python
# Butterfly pattern
n = int(input("Enter number of rows for each half: "))

for i in range(1, n + 1):
    print("*" * i + " " * (2 * (n - i)) + "*" * i)
for i in range(n - 1, 0, -1):
    print("*" * i + " " * (2 * (n - i)) + "*" * i)
```

Output:

```
Enter number of rows for each half: 5
*        *
**      **
***    ***
****  ****
**********
****  ****
***    ***
**      **
*        *
```

Program 2.61: 0-1 Triangle Pattern

Aim: To write a Python program that prints a triangle of alternating 1s and 0s.

Program:

```python
# 0-1 triangle (alternating binary pattern)
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        if (i + j) % 2 == 0:
            print(1, end=" ")
        else:
            print(0, end=" ")
    print()
```

Output:

```
Enter number of rows: 6
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
0 1 0 1 0 1
```

String Manipulation Applications

Program 2.62: String Methods Demonstration

Aim: To write a Python program that demonstrates commonly used string methods.

Program:

```python
# Demonstration of common string methods
text = input("Enter a sentence: ")

print("Original     :", repr(text))
print("strip()      :", repr(text.strip()))
clean = text.strip()
print("upper()      :", clean.upper())
print("lower()      :", clean.lower())
print("title()      :", clean.title())
print("replace()    :", clean.replace("python", "Python"))
print("find('is')   :", clean.find("is"))
print("count('o')   :", clean.count("o"))
words = clean.split()
print("split()      :", words)
print("join('-')    :", "-".join(words))
print("startswith('python'):", clean.startswith("python"))
print("endswith('fun')     :", clean.endswith("fun"))
```

Output:

```
Enter a sentence:    python is easy and python is fun
Original     : '   python is easy and python is fun   '
strip()      : 'python is easy and python is fun'
upper()      : PYTHON IS EASY AND PYTHON IS FUN
lower()      : python is easy and python is fun
title()      : Python Is Easy And Python Is Fun
replace()    : Python is easy and Python is fun
find('is')   : 7
count('o')   : 2
split()      : ['python', 'is', 'easy', 'and', 'python', 'is', 'fun']
join('-')    : python-is-easy-and-python-is-fun
startswith('python'): True
endswith('fun')     : True
```

Program 2.63: Palindrome String

Aim: To write a Python program that checks whether a given string is a palindrome.

Program:

```python
# Check whether a string is a palindrome
text = input("Enter a string: ")

cleaned = ""
for ch in text:
    if ch.isalnum():
        cleaned += ch.lower()

if cleaned == cleaned[::-1]:
    print(f'"{text}" is a Palindrome')
else:
    print(f'"{text}" is not a Palindrome')
```

Output:

```
Enter a string: Madam In Eden Im Adam
"Madam In Eden Im Adam" is a Palindrome
```

Program 2.64: Count Vowels, Consonants, Digits and Spaces

Aim: To write a Python program that counts the vowels, consonants, digits, spaces and special characters in a string.

Program:

```
# Count vowels, consonants, digits, spaces and special characters
text = input("Enter a string: ")

vowels = consonants = digits = spaces = specials = 0
for ch in text:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    elif ch.isspace():
        spaces += 1
    else:
        specials += 1

print("Vowels     :", vowels)
print("Consonants :", consonants)
print("Digits     :", digits)
print("Spaces     :", spaces)
print("Specials   :", specials)
```

Output:

```
Enter a string: Python 3.12 was released in 2023!
Vowels     : 7
Consonants : 12
Digits     : 7
Spaces     : 5
Specials   : 2
```

Program 2.65: Reverse Words in a Sentence

Aim: To write a Python program that reverses the order of words in a sentence and also reverses each word.

Program:

```
# Reverse words in a sentence
sentence = input("Enter a sentence: ")

words = sentence.split()
reversed_order = " ".join(words[::-1])
print("Words in reverse order :", reversed_order)

reversed_each = []
for word in words:
```

```
    reversed_each.append(word[::-1])
print("Each word reversed     :", " ".join(reversed_each))
```

Output:

```
Enter a sentence: Python makes programming simple
Words in reverse order : simple programming makes Python
Each word reversed     : nohtyP sekam gnimmargorp elpmis
```

Program 2.66: Character Frequency

Aim: To write a Python program that counts the frequency of each character in a string.

Program:

```
# Frequency of each character in a string
text = input("Enter a string: ")

frequency = {}
for ch in text:
    if ch != " ":
        frequency[ch] = frequency.get(ch, 0) + 1

print("Character frequencies:")
for ch, count in frequency.items():
    print(f"'{ch}' : {count}")
```

Output:

```
Enter a string: hello world
Character frequencies:
'h' : 1
'e' : 1
'l' : 3
'o' : 2
'w' : 1
'r' : 1
'd' : 1
```

Program 2.67: Anagram Check

Aim: To write a Python program that checks whether two strings are anagrams of each other.

Program:

```
# Check whether two strings are anagrams
first = input("Enter first string: ")
second = input("Enter second string: ")

s1 = first.replace(" ", "").lower()
s2 = second.replace(" ", "").lower()

if sorted(s1) == sorted(s2):
    print(f'"{first}" and "{second}" are Anagrams')
else:
    print(f'"{first}" and "{second}" are not Anagrams')
```

Output:

```
Enter first string: Listen
Enter second string: Silent
"Listen" and "Silent" are Anagrams
```

Program 2.68: Remove Duplicates and Punctuation

Aim: To write a Python program that removes punctuation marks and duplicate characters from a string.

Program:

```
# Remove punctuation and duplicate characters from a string
import string

text = input("Enter a string: ")

# Remove punctuation
no_punct = ""
for ch in text:
    if ch not in string.punctuation:
        no_punct += ch
print("Without punctuation:", no_punct)

# Remove duplicate characters (keep first occurrence)
unique = ""
for ch in no_punct:
    if ch not in unique:
        unique += ch
print("Without duplicates :", unique)
```

Output:

```
Enter a string: Hello!!! How are you, programmer?
Without punctuation: Hello How are you programmer
Without duplicates : Helo waryupgm
```

Program 2.69: String Formatting Techniques

Aim: To write a Python program that demonstrates string formatting using the % operator, format() and f-strings.

Program:

```
# String formatting with %, format() and f-strings
name = input("Enter student name: ")
roll = int(input("Enter roll number: "))
percentage = float(input("Enter percentage: "))

print("Using %% operator : Name: %s, Roll: %d, Percentage: %.2f" % (name, roll, percentag
e))
print("Using format()   : Name: {}, Roll: {}, Percentage: {:.2f}".format(name, roll, perc
entage))
print(f"Using f-string   : Name: {name}, Roll: {roll}, Percentage: {percentage:.2f}")

# Alignment examples
```

```
print(f"|{name:<12}|{roll:^6}|{percentage:>8.1f}|")
print("|{:<12}|{:^6}|{:>8.1f}|".format(name, roll, percentage))
```

Output:

```
Enter student name: Ravi Kumar
Enter roll number: 105
Enter percentage: 87.456
Using % operator : Name: Ravi Kumar, Roll: 105, Percentage: 87.46
Using format()   : Name: Ravi Kumar, Roll: 105, Percentage: 87.46
Using f-string   : Name: Ravi Kumar, Roll: 105, Percentage: 87.46
|Ravi Kumar  | 105  |    87.5|
|Ravi Kumar  | 105  |    87.5|
```

Program 2.70: Longest Word in a Sentence

Aim: To write a Python program that finds the longest word in a sentence.

Program:

```
# Find the longest word in a sentence
sentence = input("Enter a sentence: ")

words = sentence.split()
longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word

print("Longest word:", longest)
print("Length      :", len(longest))
```

Output:

```
Enter a sentence: Python supports object oriented and functional programming
Longest word: programming
Length      : 11
```

Program 2.71: Capitalise Each Word without title()

Aim: To write a Python program that capitalises the first letter of each word without using the title() method.

Program:

```
# Capitalise each word without using title()
sentence = input("Enter a sentence: ")

result = []
for word in sentence.split():
    result.append(word[0].upper() + word[1:].lower())

print("Capitalised:", " ".join(result))
```

Output:

```
Enter a sentence: welcome to PYTHON programming lab
Capitalised: Welcome To Python Programming Lab
```

Program 2.72: Toggle Case of a String

Aim: To write a Python program that toggles the case of every letter in a string.

Program:

```
# Toggle the case of each character
text = input("Enter a string: ")

toggled = ""
for ch in text:
    if ch.isupper():
        toggled += ch.lower()
    elif ch.islower():
        toggled += ch.upper()
    else:
        toggled += ch

print("Toggled (loop)      :", toggled)
print("Toggled (swapcase()):", text.swapcase())
```

Output:

```
Enter a string: Hello World 2024
Toggled (loop)      : hELLO wORLD 2024
Toggled (swapcase()): hELLO wORLD 2024
```

Program 2.73: String Slicing Demonstration

Aim: To write a Python program that demonstrates string indexing and slicing.

Program:

```
# String indexing and slicing demonstration
s = input("Enter a string: ")

print("String          :", s)
print("Length          :", len(s))
print("First character :", s[0])
print("Last character  :", s[-1])
print("s[0:4]          :", s[0:4])
print("s[4:]           :", s[4:])
print("s[:6]           :", s[:6])
print("s[-5:]          :", s[-5:])
print("s[::2]          :", s[::2])
print("s[1::3]         :", s[1::3])
print("s[::-1]         :", s[::-1])
```

Output:

```
Enter a string: PythonProgramming
String          : PythonProgramming
Length          : 17
First character : P
Last character  : g
s[0:4]          : Pyth
s[4:]           : onProgramming
s[:6]           : Python
```

```
s[-5:]          : mming
s[::2]          : PtoPormig
s[1::3]         : yorrmg
s[::-1]         : gnimmargorPnohtyP
```

Program 2.74: Acronym Generator

Aim: To write a Python program that generates an acronym from a phrase, ignoring small words like ‘of’ and ‘and’.

Program:

```
# Generate an acronym from a phrase
phrase = input("Enter a phrase: ")
skip_words = ["of", "and", "the", "for", "in", "a", "an"]

acronym = ""
for word in phrase.split():
    if word.lower() not in skip_words:
        acronym += word[0].upper()

print("Acronym:", acronym)
```

Output:

```
Enter a phrase: Indian Institute of Technology
Acronym: IIT
```

Program 2.75: Pangram Check

Aim: To write a Python program that checks whether a sentence is a pangram (contains every letter a-z) and lists missing letters.

Program:

```
# Check whether a sentence is a pangram
sentence = input("Enter a sentence: ").lower()

missing = ""
for ch in "abcdefghijklmnopqrstuvwxyz":
    if ch not in sentence:
        missing += ch

if missing == "":
    print("The sentence is a pangram")
else:
    print("Not a pangram. Missing letters:", ", ".join(missing))

sentence2 = input("Enter another sentence: ").lower()
letters = set(c for c in sentence2 if c.isalpha())
print("Pangram" if len(letters) == 26 else "Not a pangram")
```

Output:

```
Enter a sentence: The quick brown fox jumps over the lazy dog
The sentence is a pangram
```

```
Enter another sentence: Hello world from Python
Not a pangram
```

Program 2.76: Mask Card Number Showing Last 4 Digits

Aim: To write a Python program that masks a card-like number so that only the last four digits are visible.

Program:

```
# Mask a 16-digit card-like number
raw = input("Enter 16-digit card number: ")
digits = raw.replace(" ", "").replace("-", "")

if len(digits) != 16 or not digits.isdigit():
    print("Invalid card number")
else:
    masked = "X" * 12 + digits[-4:]
    groups = []
    for i in range(0, 16, 4):
        groups.append(masked[i:i + 4])
    print("Masked number:", " ".join(groups))
```

Output:

```
Enter 16-digit card number: 4532-7812-0099-6627
Masked number: XXXX XXXX XXXX 6627
```

Program 2.77: Check Rotation of Strings

Aim: To write a Python program that checks whether one string is a rotation of another.

Program:

```
# Check whether s2 is a rotation of s1
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

if len(s1) == len(s2) and s2 in s1 + s1:
    shift = (s1 + s1).find(s2)
    print(f"'{s2}' is a rotation of '{s1}' (left rotation by {shift})")
else:
    print(f"'{s2}' is NOT a rotation of '{s1}'")
```

Output:

```
Enter first string: WATERBOTTLE
Enter second string: BOTTLEWATER
'BOTTLEWATER' is a rotation of 'WATERBOTTLE' (left rotation by 5)
```

Program 2.78: Word Replace and Count Occurrences

Aim: To write a Python program that counts how many times a word occurs in a text and replaces it with another word.

Program:

```python
# Count and replace a word in a text
text = input("Enter text: ")
old = input("Word to replace: ")
new = input("Replace with: ")

count = 0
result_words = []
for word in text.split():
    core = word.strip(".,!?;:")
    if core.lower() == old.lower():
        count += 1
        word = word.replace(core, new)
    result_words.append(word)

print("Occurrences of", repr(old), ":", count)
print("Updated text:", " ".join(result_words))
```

Output:

```
Enter text: Python is easy. I like python because python is powerful!
Word to replace: python
Replace with: Java
Occurrences of 'python' : 3
Updated text: Java is easy. I like Java because Java is powerful!
```

Program 2.79: Run-Length String Compression

Aim: To write a Python program that compresses a string using run-length encoding and decompresses it back.

Program:

```python
# Run-length encoding and decoding
text = input("Enter string to compress: ")

encoded = ""
i = 0
while i < len(text):
    count = 1
    while i + count < len(text) and text[i + count] == text[i]:
        count += 1
    encoded += text[i] + str(count)
    i += count
print("Compressed:", encoded)
print("Original length:", len(text), "Compressed length:", len(encoded))

decoded = ""
i = 0
while i < len(encoded):
    ch = encoded[i]
    i += 1
    num = ""
    while i < len(encoded) and encoded[i].isdigit():
        num += encoded[i]
        i += 1
    decoded += ch * int(num)
```

```
print("Decompressed:", decoded)
print("Match:", decoded == text)
```

Output:

```
Enter string to compress: aaabbbbbbbbbbccdaaa
Compressed: a3b10c2d1a3
Original length: 19 Compressed length: 11
Decompressed: aaabbbbbbbbbbccdaaa
Match: True
```

Program 2.80: Caesar Cipher Encrypt and Decrypt

Aim: To write a Python program that encrypts and decrypts a message using the Caesar cipher with a given shift.

Program:

```
# Caesar cipher encryption and decryption
message = input("Enter message: ")
key = int(input("Enter shift key: "))


def caesar(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - ord("A") + shift) % 26 + ord("A"))
        elif ch.islower():
            result += chr((ord(ch) - ord("a") + shift) % 26 + ord("a"))
        else:
            result += ch
    return result


encrypted = caesar(message, key)
decrypted = caesar(encrypted, -key)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Output:

```
Enter message: Meet me at Gate 7, Zone B!
Enter shift key: 3
Encrypted: Phhw ph dw Jdwh 7, Crqh E!
Decrypted: Meet me at Gate 7, Zone B!
```

Program 2.81: Simple Email Validation

Aim: To write a Python program that validates email addresses using basic string rules.

Program:

```
# Simple email validation using string methods
n = int(input("How many emails to check? "))

for _ in range(n):
```

```
    email = input("Enter email: ").strip()
    reason = ""
    if " " in email:
        reason = "contains spaces"
    elif email.count("@") != 1:
        reason = "must contain exactly one @"
    else:
        user, domain = email.split("@")
        if user == "":
            reason = "user name is empty"
        elif "." not in domain or domain.startswith(".") or domain.endswith("."):
            reason = "invalid domain"
        else:
            extension = domain.split(".")[-1]
            if len(extension) < 2 or not extension.isalpha():
                reason = "invalid domain extension"
    if reason:
        print(email, "-> Invalid:", reason)
    else:
        print(email, "-> Valid")
```

Output:

```
How many emails to check? 4
Enter email: ravi.kumar@gmail.com
ravi.kumar@gmail.com -> Valid
Enter email: sita@@yahoo.com
sita@@yahoo.com -> Invalid: must contain exactly one @
Enter email: hello@domain
hello@domain -> Invalid: invalid domain
Enter email: admin@college.ac.in
admin@college.ac.in -> Valid
```

Program 2.82: Username Rules Validation

Aim: To write a Python program that checks whether a username follows given rules and reports every violation.

Program:

```
# Validate usernames against rules
n = int(input("Enter number of usernames: "))

for _ in range(n):
    name = input("Enter username: ")
    errors = []
    if not 5 <= len(name) <= 15:
        errors.append("length must be 5-15")
    if name == "" or not name[0].isalpha():
        errors.append("must start with a letter")
    for ch in name:
        if not (ch.isalnum() or ch == "_"):
            errors.append("only letters, digits and _ allowed")
            break
    if "__" in name:
        errors.append("no consecutive underscores")
    if errors:
        print(name, "-> Rejected:", "; ".join(errors))
```

```
    else:
        print(name, "-> Accepted")
```

Output:

```
Enter number of usernames: 3
Enter username: student_2026
student_2026 -> Accepted
Enter username: 9lives
9lives -> Rejected: must start with a letter
Enter username: ab__c!
ab__c! -> Rejected: only letters, digits and _ allowed; no consecutive underscores
```

Program 2.83: Password Strength Checker

Aim: To write a Python program that checks the strength of a password and suggests improvements.

Program:

```
# Password strength checker
password = input("Enter password: ")

has_upper = has_lower = has_digit = has_special = False
for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    else:
        has_special = True

score = 0
tips = []
for ok, tip in [(len(password) >= 8, "use at least 8 characters"),
                (has_upper, "add an uppercase letter"),
                (has_lower, "add a lowercase letter"),
                (has_digit, "add a digit"),
                (has_special, "add a special character")]:
    if ok:
        score += 1
    else:
        tips.append(tip)

if score == 5:
    strength = "Strong"
elif score >= 3:
    strength = "Medium"
else:
    strength = "Weak"
print("Score:", score, "/ 5 ->", strength)
for tip in tips:
    print("Suggestion:", tip)
```

Output:

```
Enter password: Python2026
Score: 4 / 5 -> Medium
Suggestion: add a special character
```

Program 2.84: Count Lines, Words and Characters of Multi-line Text

Aim: To write a Python program that reads multi-line text until END is entered and counts lines, words and characters.

Program:

```python
# Count lines, words and characters in multi-line input
print("Enter text (type END on a new line to finish):")
lines = words = chars = 0
longest = ""

while True:
    line = input()
    if line == "END":
        break
    lines += 1
    words += len(line.split())
    chars += len(line)
    if len(line) > len(longest):
        longest = line

print("Lines      :", lines)
print("Words      :", words)
print("Characters :", chars, "(excluding newlines)")
print("Longest line:", longest)
```

Output:

```
Enter text (type END on a new line to finish):
Python is a high-level language.
It supports loops and strings.

Practice makes a programmer perfect.
END
Lines      : 4
Words      : 15
Characters : 98 (excluding newlines)
Longest line: Practice makes a programmer perfect.
```

Unit III: Lists, Tuples, Sets and Functions

List Operations and Applications

Program 3.1: Basic List Operations

Aim: To write a Python program that demonstrates creation, traversal, updating, append, insert, remove, pop, sort and reverse on a list.

Program:

```
# Demonstration of common list operations
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
print("Original list:", numbers)

# Traversal using a for loop
print("Traversal:", end=" ")
for item in numbers:
    print(item, end=" ")
print()

# Updating an element by index
numbers[0] = 100
print("After updating index 0 to 100:", numbers)

numbers.append(50)
print("After append(50):", numbers)

numbers.insert(2, 25)
print("After insert(2, 25):", numbers)

numbers.remove(25)
print("After remove(25):", numbers)

popped = numbers.pop()
print("Popped element:", popped, "| List now:", numbers)

numbers.sort()
print("After sort():", numbers)

numbers.reverse()
print("After reverse():", numbers)

print("Length of list:", len(numbers))
```

Output:

```
Enter numbers separated by space: 12 7 45 3 19
Original list: [12, 7, 45, 3, 19]
Traversal: 12 7 45 3 19
After updating index 0 to 100: [100, 7, 45, 3, 19]
After append(50): [100, 7, 45, 3, 19, 50]
After insert(2, 25): [100, 7, 25, 45, 3, 19, 50]
After remove(25): [100, 7, 45, 3, 19, 50]
```

```
Popped element: 50 | List now: [100, 7, 45, 3, 19]
After sort(): [3, 7, 19, 45, 100]
After reverse(): [100, 45, 19, 7, 3]
Length of list: 5
```

Program 3.2: Largest and Smallest Element without max/min

Aim: To write a Python program that finds the largest and smallest elements of a list without using max() and min().

Program:

```
# Find largest and smallest without built-in functions
values = list(map(int, input("Enter numbers separated by space: ").split()))

largest = values[0]
smallest = values[0]
for num in values[1:]:
    if num > largest:
        largest = num
    if num < smallest:
        smallest = num

print("List:", values)
print("Largest element:", largest)
print("Smallest element:", smallest)
```

Output:

```
Enter numbers separated by space: 34 12 89 5 67 23
List: [34, 12, 89, 5, 67, 23]
Largest element: 89
Smallest element: 5
```

Program 3.3: Sum, Average and Second Largest

Aim: To write a Python program that finds the sum, average and second largest element of a list.

Program:

```
# Sum, average and second largest element of a list
marks = list(map(int, input("Enter numbers separated by space: ").split()))

total = 0
for m in marks:
    total += m
average = total / len(marks)

first = second = None
for m in marks:
    if first is None or m > first:
        second = first
        first = m
    elif m != first and (second is None or m > second):
        second = m
```

```
print("Sum =", total)
print("Average = %.2f" % average)
if second is None:
    print("No second largest element (all values equal)")
else:
    print("Second largest =", second)
```

Output:

```
Enter numbers separated by space: 45 78 92 78 60 92
Sum = 445
Average = 74.17
Second largest = 78
```

Program 3.4: Remove Duplicates and Count Frequency

Aim: To write a Python program that removes duplicate elements from a list preserving order and counts the frequency of each element.

Program:

```
# Remove duplicates (order preserved) and find frequency
items = input("Enter elements separated by space: ").split()

unique_items = []
for item in items:
    if item not in unique_items:
        unique_items.append(item)

frequency = {}
for item in items:
    frequency[item] = frequency.get(item, 0) + 1

print("Original list:", items)
print("Without duplicates:", unique_items)
print("Frequency of elements:")
for item in unique_items:
    print(" ", item, "->", frequency[item])
```

Output:

```
Enter elements separated by space: apple mango apple kiwi mango apple banana
Original list: ['apple', 'mango', 'apple', 'kiwi', 'mango', 'apple', 'banana']
Without duplicates: ['apple', 'mango', 'kiwi', 'banana']
Frequency of elements:
  apple -> 3
  mango -> 2
  kiwi -> 1
  banana -> 1
```

Program 3.5: Linear Search

Aim: To write a Python program that searches for an element in a list using linear search.

Program:

```
# Linear search in a list
def linear_search(data, key):
```

```
    """Return the index of key in data, or -1 if not found."""
    for index in range(len(data)):
        if data[index] == key:
            return index
    return -1


numbers = list(map(int, input("Enter numbers separated by space: ").split()))
key = int(input("Enter element to search: "))

position = linear_search(numbers, key)
if position == -1:
    print(key, "not found in the list")
else:
    print(key, "found at index", position, "(position", position + 1, ")")
```

Output:

```
Enter numbers separated by space: 15 42 8 23 56 4
Enter element to search: 23
23 found at index 3 (position 4 )
```

Program 3.6: Bubble Sort

Aim: To write a Python program that sorts a list in ascending order using bubble sort.

Program:

```
# Bubble sort with early exit
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
n = len(numbers)

for i in range(n - 1):
    swapped = False
    for j in range(n - 1 - i):
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
            swapped = True
    print("After pass", i + 1, ":", numbers)
    if not swapped:
        break

print("Sorted list:", numbers)
```

Output:

```
Enter numbers separated by space: 64 34 25 12 22 11 90
After pass 1 : [34, 25, 12, 22, 11, 64, 90]
After pass 2 : [25, 12, 22, 11, 34, 64, 90]
After pass 3 : [12, 22, 11, 25, 34, 64, 90]
After pass 4 : [12, 11, 22, 25, 34, 64, 90]
After pass 5 : [11, 12, 22, 25, 34, 64, 90]
After pass 6 : [11, 12, 22, 25, 34, 64, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

Program 3.7: List Comprehension and Even/Odd Separation

Aim: To write a Python program that uses list comprehensions to build squares and even numbers and to separate even and odd elements.

Program:

```
# List comprehension examples
n = int(input("Enter n: "))

squares = [x * x for x in range(1, n + 1)]
evens = [x for x in range(1, n + 1) if x % 2 == 0]
print("Squares of 1 to", n, ":", squares)
print("Even numbers from 1 to", n, ":", evens)

values = list(map(int, input("Enter numbers separated by space: ").split()))
even_list = [v for v in values if v % 2 == 0]
odd_list = [v for v in values if v % 2 != 0]
print("Even elements:", even_list)
print("Odd elements:", odd_list)
```

Output:

```
Enter n: 10
Squares of 1 to 10 : [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even numbers from 1 to 10 : [2, 4, 6, 8, 10]
Enter numbers separated by space: 11 24 35 46 57 68 79
Even elements: [24, 46, 68]
Odd elements: [11, 35, 57, 79]
```

Program 3.8: Rotate a List

Aim: To write a Python program that rotates a list left and right by k positions.

Program:

```
# Left and right rotation of a list using slicing
data = list(map(int, input("Enter numbers separated by space: ").split()))
k = int(input("Enter number of positions to rotate: "))

k = k % len(data)
left_rotated = data[k:] + data[:k]
right_rotated = data[-k:] + data[:-k] if k else data[:]

print("Original list:", data)
print("Left rotated by", k, ":", left_rotated)
print("Right rotated by", k, ":", right_rotated)
```

Output:

```
Enter numbers separated by space: 1 2 3 4 5 6 7
Enter number of positions to rotate: 2
Original list: [1, 2, 3, 4, 5, 6, 7]
Left rotated by 2 : [3, 4, 5, 6, 7, 1, 2]
Right rotated by 2 : [6, 7, 1, 2, 3, 4, 5]
```

Program 3.9: Matrix Addition, Multiplication and Transpose

Aim: To write a Python program that performs addition, multiplication and transpose of matrices using nested lists.

Program:

```
# Matrix operations using nested lists
def read_matrix(name, rows):
    """Read a matrix row by row from the user."""
    print("Enter", rows, "rows of matrix", name, "(space separated):")
    return [list(map(int, input().split())) for _ in range(rows)]


def print_matrix(title, matrix):
    print(title)
    for row in matrix:
        print(" ".join(str(x).rjust(4) for x in row))


n = int(input("Enter order of square matrices: "))
a = read_matrix("A", n)
b = read_matrix("B", n)

addition = [[a[i][j] + b[i][j] for j in range(n)] for i in range(n)]

product = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        for k in range(n):
            product[i][j] += a[i][k] * b[k][j]

transpose = [[a[j][i] for j in range(n)] for i in range(n)]

print_matrix("A + B:", addition)
print_matrix("A x B:", product)
print_matrix("Transpose of A:", transpose)
```

Output:

```
Enter order of square matrices: 2
Enter 2 rows of matrix A (space separated):
1 2
3 4
Enter 2 rows of matrix B (space separated):
5 6
7 8
A + B:
   6    8
  10   12
A x B:
  19   22
  43   50
Transpose of A:
   1    3
   2    4
```

Program 3.10: Student Marks Analysis using a List

Aim: To write a Python program that stores the marks of a class in a list and finds the total, average, highest, lowest and pass count.

Program:

```
# Class marks analysis using a list
count = int(input("Enter number of students: "))
marks = []
for i in range(count):
    marks.append(int(input("Enter marks of student " + str(i + 1) + ": ")))

PASS_MARK = 40
total = sum(marks)
average = total / len(marks)

pass_count = 0
for m in marks:
    if m >= PASS_MARK:
        pass_count += 1

print("Marks list    :", marks)
print("Total marks   :", total)
print("Class average : %.2f" % average)
print("Highest marks :", max(marks))
print("Lowest marks  :", min(marks))
print("Passed        :", pass_count)
print("Failed        :", len(marks) - pass_count)
print("Pass percentage: %.1f%%" % (pass_count * 100 / len(marks)))
```

Output:

```
Enter number of students: 6
Enter marks of student 1: 78
Enter marks of student 2: 35
Enter marks of student 3: 92
Enter marks of student 4: 61
Enter marks of student 5: 28
Enter marks of student 6: 55
Marks list    : [78, 35, 92, 61, 28, 55]
Total marks   : 349
Class average : 58.17
Highest marks : 92
Lowest marks  : 28
Passed        : 4
Failed        : 2
Pass percentage: 66.7%
```

Program 3.11: Menu-Driven Shopping List Manager

Aim: To write a Python program that manages a shopping list with a menu to add, remove and view items until the user exits.

Program:

```python
# Shopping list manager using a list
shopping = []

while True:
    print("1.Add item 2.Remove item 3.View list 4.Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        item = input("Item to add: ").strip().title()
        if item in shopping:
            print(item, "is already in the list")
        else:
            shopping.append(item)
            print(item, "added")
    elif choice == "2":
        item = input("Item to remove: ").strip().title()
        if item in shopping:
            shopping.remove(item)
            print(item, "removed")
        else:
            print(item, "not found in the list")
    elif choice == "3":
        if not shopping:
            print("Shopping list is empty")
        else:
            print("Shopping list (" + str(len(shopping)) + " items):")
            for pos, item in enumerate(shopping, start=1):
                print(" ", pos, item)
    elif choice == "4":
        print("Goodbye! Final list:", shopping)
        break
    else:
        print("Invalid choice")
```

Output:

```
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 1
Item to add: rice
Rice added
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 1
Item to add: milk
Milk added
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 1
Item to add: sugar
Sugar added
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 1
Item to add: Milk
Milk is already in the list
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 3
Shopping list (3 items):
  1 Rice
  2 Milk
  3 Sugar
1.Add item 2.Remove item 3.View list 4.Exit
```

```
Enter choice: 2
Item to remove: sugar
Sugar removed
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 2
Item to remove: oil
Oil not found in the list
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 3
Shopping list (2 items):
  1 Rice
  2 Milk
1.Add item 2.Remove item 3.View list 4.Exit
Enter choice: 4
Goodbye! Final list: ['Rice', 'Milk']
```

Program 3.12: Cumulative (Running) Sum of Daily Sales

Aim: To write a Python program that finds the cumulative sum of daily sales stored in a list.

Program:

```
# Cumulative sum of daily sales
sales = list(map(int, input("Enter daily sales separated by space: ").split()))

running_total = 0
cumulative = []
for amount in sales:
    running_total += amount
    cumulative.append(running_total)

print("Daily sales     :", sales)
print("Cumulative sales:", cumulative)
for day in range(len(sales)):
    print("Day", day + 1, "-> sale", sales[day], "| total so far", cumulative[day])
```

Output:

```
Enter daily sales separated by space: 1200 950 1430 800 1600
Daily sales     : [1200, 950, 1430, 800, 1600]
Cumulative sales: [1200, 2150, 3580, 4380, 5980]
Day 1 -> sale 1200 | total so far 1200
Day 2 -> sale 950 | total so far 2150
Day 3 -> sale 1430 | total so far 3580
Day 4 -> sale 800 | total so far 4380
Day 5 -> sale 1600 | total so far 5980
```

Program 3.13: Temperature Readings - Days Above Average

Aim: To write a Python program that reads a week's temperatures and lists the days on which the temperature was above the average.

Program:

```
# Days with temperature above the weekly average
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
temps = []
```

```
for day in days:
    temps.append(float(input("Temperature on " + day + ": ")))

average = sum(temps) / len(temps)
above = []
for i in range(len(temps)):
    if temps[i] > average:
        above.append(days[i])

print("Average temperature: %.2f" % average)
print("Days above average :", above)
print("Hottest day:", days[temps.index(max(temps))], "-", max(temps))
print("Coolest day:", days[temps.index(min(temps))], "-", min(temps))
```

Output:

```
Temperature on Mon: 31.5
Temperature on Tue: 33.0
Temperature on Wed: 29.8
Temperature on Thu: 35.2
Temperature on Fri: 34.1
Temperature on Sat: 30.0
Temperature on Sun: 32.4
Average temperature: 32.29
Days above average : ['Tue', 'Thu', 'Fri', 'Sun']
Hottest day: Thu - 35.2
Coolest day: Wed - 29.8
```

Program 3.14: Count Positive, Negative and Zero Values

Aim: To write a Python program that counts and separates positive, negative and zero values in a list of bank transactions.

Program:

```
# Classify transactions as credit, debit or zero
transactions = list(map(int, input("Enter transactions: ").split()))

credits, debits, zeros = [], [], []
for amount in transactions:
    if amount > 0:
        credits.append(amount)
    elif amount < 0:
        debits.append(amount)
    else:
        zeros.append(amount)

print("Positive (credits):", credits, "count =", len(credits))
print("Negative (debits) :", debits, "count =", len(debits))
print("Zero entries      :", len(zeros))
print("Net balance change:", sum(transactions))
```

Output:

```
Enter transactions: 5000 -1200 0 -350 2500 0 -800
Positive (credits): [5000, 2500] count = 2
Negative (debits) : [-1200, -350, -800] count = 3
```

```
Zero entries      : 2
Net balance change: 5150
```

Program 3.15: Merge Two Sorted Lists without sort()

Aim: To write a Python program that merges two sorted lists into one sorted list without using sort() or sorted().

Program:

```
# Merge two sorted lists (merge step of merge sort)
a = list(map(int, input("Enter first sorted list: ").split()))
b = list(map(int, input("Enter second sorted list: ").split()))

merged = []
i = j = 0
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        merged.append(a[i])
        i += 1
    else:
        merged.append(b[j])
        j += 1

# Copy whatever is left
merged.extend(a[i:])
merged.extend(b[j:])

print("Merged sorted list:", merged)
```

Output:

```
Enter first sorted list: 3 8 15 22 40
Enter second sorted list: 1 9 10 30
Merged sorted list: [1, 3, 8, 9, 10, 15, 22, 30, 40]
```

Program 3.16: Find All Pairs with a Given Sum

Aim: To write a Python program that finds all pairs of elements in a list whose sum equals a given target.

Program:

```
# All pairs whose sum equals a target (e.g. bills matching a budget)
prices = list(map(int, input("Enter item prices: ").split()))
target = int(input("Enter target amount: "))

pairs = []
for i in range(len(prices)):
    for j in range(i + 1, len(prices)):
        if prices[i] + prices[j] == target:
            pairs.append((prices[i], prices[j]))

if pairs:
    print("Pairs adding up to", target, ":")
    for p in pairs:
```

```
        print(" ", p[0], "+", p[1])
else:
    print("No pair adds up to", target)
```

Output:

```
Enter item prices: 250 400 150 350 600 100 500
Enter target amount: 750
Pairs adding up to 750 :
  250 + 500
  400 + 350
  150 + 600
```

Program 3.17: Move All Zeros to the End

Aim: To write a Python program that moves all zeros of a list to the end while keeping the order of the non-zero elements.

Program:

```
# Move zeros to the end (in place)
values = list(map(int, input("Enter numbers: ").split()))
print("Original list:", values)

pos = 0
for num in values:
    if num != 0:
        values[pos] = num
        pos += 1
while pos < len(values):
    values[pos] = 0
    pos += 1

print("After moving zeros:", values)
```

Output:

```
Enter numbers: 0 4 0 7 2 0 0 9 1
Original list: [0, 4, 0, 7, 2, 0, 0, 9, 1]
After moving zeros: [4, 7, 2, 9, 1, 0, 0, 0, 0]
```

Program 3.18: Insertion Sort

Aim: To write a Python program that sorts a list of numbers using the insertion sort technique.

Program:

```
# Insertion sort
data = list(map(int, input("Enter numbers to sort: ").split()))

for i in range(1, len(data)):
    key = data[i]
    j = i - 1
    while j >= 0 and data[j] > key:
        data[j + 1] = data[j]      # shift right
        j -= 1
    data[j + 1] = key
    print("Pass", i, ":", data)
```

```
print("Sorted list:", data)
```

Output:

```
Enter numbers to sort: 29 10 14 37 13 5
Pass 1 : [10, 29, 14, 37, 13, 5]
Pass 2 : [10, 14, 29, 37, 13, 5]
Pass 3 : [10, 14, 29, 37, 13, 5]
Pass 4 : [10, 13, 14, 29, 37, 5]
Pass 5 : [5, 10, 13, 14, 29, 37]
Sorted list: [5, 10, 13, 14, 29, 37]
```

Program 3.19: Selection Sort

Aim: To write a Python program that sorts a list of numbers using the selection sort technique.

Program:

```
# Selection sort
data = list(map(int, input("Enter numbers to sort: ").split()))
n = len(data)

for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if data[j] < data[min_index]:
            min_index = j
    data[i], data[min_index] = data[min_index], data[i]
    print("Pass", i + 1, ":", data)

print("Sorted list:", data)
```

Output:

```
Enter numbers to sort: 64 25 12 22 11
Pass 1 : [11, 25, 12, 22, 64]
Pass 2 : [11, 12, 25, 22, 64]
Pass 3 : [11, 12, 22, 25, 64]
Pass 4 : [11, 12, 22, 25, 64]
Sorted list: [11, 12, 22, 25, 64]
```

Program 3.20: Stack Implementation using a List

Aim: To write a Python program that implements a stack (push, pop, peek, display) using a list with a menu.

Program:

```
# Stack using a list (LIFO)
stack = []

while True:
    print("1.Push 2.Pop 3.Peek 4.Display 5.Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        item = input("Enter element: ")
```

```
        stack.append(item)
        print("Pushed", item)
    elif choice == "2":
        if stack:
            print("Popped", stack.pop())
        else:
            print("Stack underflow - stack is empty")
    elif choice == "3":
        print("Top element:", stack[-1] if stack else "None (empty)")
    elif choice == "4":
        print("Stack (top -> bottom):", stack[::-1])
    elif choice == "5":
        print("Exit")
        break
    else:
        print("Invalid choice")
```

Output:

```
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 10
Pushed 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 20
Pushed 20
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 30
Pushed 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top -> bottom): ['30', '20', '10']
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 3
Top element: 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Popped 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top -> bottom): ['20', '10']
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Popped 20
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Popped 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Stack underflow - stack is empty
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 5
Exit
```

Program 3.21: Queue Implementation using a List

Aim: To write a Python program that simulates a ticket-counter queue (enqueue, dequeue, display) using a list.

Program:

```
# Ticket counter queue using a list (FIFO)
queue = []

while True:
    print("1.Join queue 2.Serve 3.Show queue 4.Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        name = input("Customer name: ")
        queue.append(name)
        print(name, "joined. Position:", len(queue))
    elif choice == "2":
        if queue:
            print("Serving", queue.pop(0))
        else:
            print("Queue is empty")
    elif choice == "3":
        print("Front ->", queue, "<- Rear")
    elif choice == "4":
        print("Counter closed.", len(queue), "customer(s) still waiting")
        break
    else:
        print("Invalid choice")
```

Output:

```
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 1
Customer name: Ravi
Ravi joined. Position: 1
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 1
Customer name: Sita
Sita joined. Position: 2
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 1
Customer name: Kiran
Kiran joined. Position: 3
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 3
Front -> ['Ravi', 'Sita', 'Kiran'] <- Rear
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 2
Serving Ravi
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 2
Serving Sita
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 3
Front -> ['Kiran'] <- Rear
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 1
```

```
Customer name: Anil
Anil joined. Position: 2
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 3
Front -> ['Kiran', 'Anil'] <- Rear
1.Join queue 2.Serve 3.Show queue 4.Exit
Enter choice: 4
Counter closed. 2 customer(s) still waiting
```

Tuple Manipulation Programs

Program 3.22: Tuple Creation, Indexing and Slicing

Aim: To write a Python program that creates a tuple and demonstrates indexing, slicing, count() and index().

Program:

```
# Tuple creation and access
values = tuple(map(int, input("Enter numbers separated by space: ").split()))
single = (5,)          # a one-element tuple needs a trailing comma
empty = ()

print("Tuple:", values)
print("Single element tuple:", single, "| Empty tuple:", empty)
print("First element:", values[0])
print("Last element:", values[-1])
print("Slice [1:4]:", values[1:4])
print("Reversed using [::-1]:", values[::-1])

key = int(input("Enter element to count and locate: "))
print("Count of", key, ":", values.count(key))
if key in values:
    print("First index of", key, ":", values.index(key))
else:
    print(key, "is not in the tuple")
```

Output:

```
Enter numbers separated by space: 10 20 30 20 40 20 50
Tuple: (10, 20, 30, 20, 40, 20, 50)
Single element tuple: (5,) | Empty tuple: ()
First element: 10
Last element: 50
Slice [1:4]: (20, 30, 20)
Reversed using [::-1]: (50, 20, 40, 20, 30, 20, 10)
Enter element to count and locate: 20
Count of 20 : 3
First index of 20 : 1
```

Program 3.23: Tuple Packing, Unpacking and Swapping

Aim: To write a Python program that demonstrates tuple packing, unpacking (including star unpacking) and swapping two variables.

Program:

```
# Packing, unpacking and swapping with tuples
student = "Ravi", 21, "CSE"          # packing
name, age, branch = student          # unpacking
print("Packed tuple:", student)
print("Name:", name, "| Age:", age, "| Branch:", branch)

scores = (90, 85, 78, 66, 54)
first, *middle, last = scores        # star unpacking
print("First:", first, "| Middle:", middle, "| Last:", last)

a = int(input("Enter a: "))
b = int(input("Enter b: "))
print("Before swap: a =", a, ", b =", b)
a, b = b, a
print("After swap:  a =", a, ", b =", b)
```

Output:

```
Packed tuple: ('Ravi', 21, 'CSE')
Name: Ravi | Age: 21 | Branch: CSE
First: 90 | Middle: [85, 78, 66] | Last: 54
Enter a: 15
Enter b: 40
Before swap: a = 15 , b = 40
After swap:  a = 40 , b = 15
```

Program 3.24: List-Tuple Conversion and Immutability

Aim: To write a Python program that converts between lists and tuples and shows that tuples are immutable.

Program:

```
# Converting list <-> tuple and immutability
colours = input("Enter colours separated by space: ").split()
colour_tuple = tuple(colours)
print("List :", colours)
print("Tuple:", colour_tuple)

try:
    colour_tuple[0] = "black"
except TypeError as error:
    print("Error while modifying tuple:", error)

# To change a tuple, convert to list, modify, convert back
temp = list(colour_tuple)
temp[0] = "black"
colour_tuple = tuple(temp)
print("Modified tuple (via list):", colour_tuple)
```

Output:

```
Enter colours separated by space: red green blue
List : ['red', 'green', 'blue']
Tuple: ('red', 'green', 'blue')
Error while modifying tuple: 'tuple' object does not support item assignment
Modified tuple (via list): ('black', 'green', 'blue')
```

Program 3.25: Sort Tuples by Second Element and Tuple Statistics

Aim: To write a Python program that sorts a list of tuples by the second element and finds max, min and sum of a tuple.

Program:

```
# Sort list of tuples by second element; tuple statistics
n = int(input("Enter number of items: "))
items = []
for _ in range(n):
    name, price = input("Enter item name and price: ").split()
    items.append((name, int(price)))

by_price = sorted(items, key=lambda pair: pair[1])
print("Sorted by price:", by_price)

prices = tuple(p for _, p in items)
print("Prices tuple:", prices)
print("Maximum:", max(prices))
print("Minimum:", min(prices))
print("Sum:", sum(prices))
print("Average: %.2f" % (sum(prices) / len(prices)))
```

Output:

```
Enter number of items: 4
Enter item name and price: pen 20
Enter item name and price: book 150
Enter item name and price: bag 500
Enter item name and price: eraser 5
Sorted by price: [('eraser', 5), ('pen', 20), ('book', 150), ('bag', 500)]
Prices tuple: (20, 150, 500, 5)
Maximum: 500
Minimum: 5
Sum: 675
Average: 168.75
```

Program 3.26: Nested Tuple of Student Records

Aim: To write a Python program that stores student records in a nested tuple and displays them with the class topper.

Program:

```
# Student records stored as a nested tuple
n = int(input("Enter number of students: "))
records = ()
for _ in range(n):
    roll, name, marks = input("Enter roll, name and marks: ").split()
    records += ((int(roll), name, int(marks)),)

print()
print("Roll".ljust(6), "Name".ljust(10), "Marks")
for roll, name, marks in records:
    print(str(roll).ljust(6), name.ljust(10), marks)
```

```
topper = max(records, key=lambda rec: rec[2])
print("Topper:", topper[1], "with", topper[2], "marks")
print("Name of second student:", records[1][1])
```

Output:

```
Enter number of students: 3
Enter roll, name and marks: 101 Anil 78
Enter roll, name and marks: 102 Bhavya 91
Enter roll, name and marks: 103 Chandu 85

Roll   Name       Marks
101    Anil       78
102    Bhavya     91
103    Chandu     85
Topper: Bhavya with 91 marks
Name of second student: Bhavya
```

Program 3.27: Employee Records Sorted by Salary

Aim: To write a Python program that stores employee records as tuples (name, department, salary), sorts them by salary and finds the highest paid employee.

Program:

```
# Employee records as tuples
n = int(input("Enter number of employees: "))
employees = []
for _ in range(n):
    name, dept, salary = input("Enter name, dept, salary: ").split(",")
    employees.append((name.strip(), dept.strip(), int(salary)))

by_salary = sorted(employees, key=lambda e: e[2], reverse=True)
print("%-10s %-8s %8s" % ("Name", "Dept", "Salary"))
for name, dept, salary in by_salary:          # tuple unpacking
    print("%-10s %-8s %8d" % (name, dept, salary))

top = max(employees, key=lambda e: e[2])
print("Highest paid:", top[0], "from", top[1], "earning", top[2])
total = sum(e[2] for e in employees)
print("Total salary bill:", total)
```

Output:

```
Enter number of employees: 5
Enter name, dept, salary: Anitha, CSE, 72000
Enter name, dept, salary: Rahul, ECE, 58000
Enter name, dept, salary: Meena, CSE, 85000
Enter name, dept, salary: Suresh, MECH, 49000
Enter name, dept, salary: Farah, IT, 66000
Name       Dept       Salary
Meena      CSE         85000
Anitha     CSE         72000
Farah      IT          66000
Rahul      ECE         58000
Suresh     MECH        49000
```

```
Highest paid: Meena from CSE earning 85000
Total salary bill: 330000
```

Program 3.28: Product Catalogue - Total Stock Value

Aim: To write a Python program that stores products as (code, name, price, quantity) tuples and computes each item's and the total stock value.

Program:

```
# Inventory catalogue using tuples
n = int(input("Number of products: "))
catalogue = []
for _ in range(n):
    code, name, price, qty = input("code name price qty: ").split()
    catalogue.append((code, name, float(price), int(qty)))

grand_total = 0
print("%-6s %-10s %8s %5s %10s" % ("Code", "Item", "Price", "Qty", "Value"))
for code, name, price, qty in catalogue:
    value = price * qty
    grand_total += value
    print("%-6s %-10s %8.2f %5d %10.2f" % (code, name, price, qty, value))
print("Total stock value: %.2f" % grand_total)

low_stock = [p[1] for p in catalogue if p[3] < 10]
print("Low stock (qty < 10):", low_stock)
```

Output:

```
Number of products: 4
code name price qty: P101 Pen 10.50 120
code name price qty: P102 Notebook 45 60
code name price qty: P103 Stapler 150 8
code name price qty: P104 Marker 35 5
Code   Item          Price   Qty      Value
P101   Pen           10.50   120    1260.00
P102   Notebook      45.00    60    2700.00
P103   Stapler      150.00     8    1200.00
P104   Marker        35.00     5     175.00
Total stock value: 5335.00
Low stock (qty < 10): ['Stapler', 'Marker']
```

Program 3.29: Coordinates - Distance from Origin and Nearest Point

Aim: To write a Python program that stores points as (x, y) tuples, finds each point's distance from the origin and the nearest point.

Program:

```
# Points as tuples - distance from origin
import math

n = int(input("Enter number of points: "))
points = []
for _ in range(n):
```

```
    x, y = map(float, input("Enter x y: ").split())
    points.append((x, y))


def distance(point):
    x, y = point
    return math.sqrt(x * x + y * y)


for p in points:
    print("Point", p, "-> distance %.3f" % distance(p))

nearest = min(points, key=distance)
farthest = max(points, key=distance)
print("Nearest point to origin :", nearest)
print("Farthest point to origin:", farthest)
```

Output:

```
Enter number of points: 4
Enter x y: 3 4
Enter x y: -1 2
Enter x y: 6 -8
Enter x y: 0.5 0.5
Point (3.0, 4.0) -> distance 5.000
Point (-1.0, 2.0) -> distance 2.236
Point (6.0, -8.0) -> distance 10.000
Point (0.5, 0.5) -> distance 0.707
Nearest point to origin : (0.5, 0.5)
Farthest point to origin: (6.0, -8.0)
```

Program 3.30: Subject-wise Grades from a Tuple of Marks

Aim: To write a Python program that stores subject names and marks in tuples and prints the grade for each subject.

Program:

```
# Grade per subject using tuples
subjects = ("Maths", "Physics", "Chemistry", "English", "Python")
marks = tuple(map(int, input("Enter marks of 5 subjects: ").split()))


def grade(mark):
    if mark >= 90:
        return "O"
    elif mark >= 80:
        return "A+"
    elif mark >= 70:
        return "A"
    elif mark >= 60:
        return "B+"
    elif mark >= 50:
        return "B"
    elif mark >= 40:
        return "C"
```

```
    return "F"


for subject, mark in zip(subjects, marks):
    print("%-10s %3d  Grade: %s" % (subject, mark, grade(mark)))
print("Total:", sum(marks), "| Percentage: %.2f" % (sum(marks) / len(marks)))
print("Best subject:", subjects[marks.index(max(marks))])
```

Output:

```
Enter marks of 5 subjects: 92 78 65 84 38
Maths       92  Grade: O
Physics     78  Grade: A
Chemistry   65  Grade: B+
English     84  Grade: A+
Python      38  Grade: F
Total: 357 | Percentage: 71.40
Best subject: Maths
```

Program 3.31: Find All Occurrences of an Element in a Tuple

Aim: To write a Python program that finds the count and all index positions of a given element in a tuple.

Program:

```
# All positions of an element in a tuple
data = tuple(input("Enter tuple elements: ").split())
key = input("Element to search: ")

positions = [i for i, item in enumerate(data) if item == key]
print("Tuple:", data)
print("Count of", key, ":", data.count(key))
if positions:
    print("First occurrence (index()):", data.index(key))
    print("All positions:", positions)
else:
    print(key, "is not present in the tuple")
```

Output:

```
Enter tuple elements: red blue green red yellow red blue
Element to search: red
Tuple: ('red', 'blue', 'green', 'red', 'yellow', 'red', 'blue')
Count of red : 3
First occurrence (index()): 0
All positions: [0, 3, 5]
```

Program 3.32: Zip Two Tuples into Pairs

Aim: To write a Python program that combines a tuple of roll numbers and a tuple of names into pairs and then separates them again.

Program:

```
# Zipping and unzipping tuples
rolls = tuple(map(int, input("Enter roll numbers: ").split()))
```

```
names = tuple(input("Enter names: ").split())

pairs = tuple(zip(rolls, names))
print("Paired tuple:", pairs)
for roll, name in pairs:
    print("Roll", roll, "->", name)

unzipped_rolls, unzipped_names = zip(*pairs)
print("Unzipped rolls:", unzipped_rolls)
print("Unzipped names:", unzipped_names)
```

Output:

```
Enter roll numbers: 101 102 103 104
Enter names: Asha Bharat Chitra Deepak
Paired tuple: ((101, 'Asha'), (102, 'Bharat'), (103, 'Chitra'), (104, 'Deepak'))
Roll 101 -> Asha
Roll 102 -> Bharat
Roll 103 -> Chitra
Roll 104 -> Deepak
Unzipped rolls: (101, 102, 103, 104)
Unzipped names: ('Asha', 'Bharat', 'Chitra', 'Deepak')
```

Program 3.33: Date Tuple Validation and Formatting

Aim: To write a Python program that validates a date stored as a (day, month, year) tuple and prints it in different formats.

Program:

```
# Validate and format a (day, month, year) tuple
MONTHS = ("January", "February", "March", "April", "May", "June", "July",
          "August", "September", "October", "November", "December")
DAYS = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)


def is_leap(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)


def is_valid(date):
    day, month, year = date
    if month < 1 or month > 12 or year < 1:
        return False
    limit = DAYS[month - 1]
    if month == 2 and is_leap(year):
        limit = 29
    return 1 <= day <= limit


for _ in range(3):
    date = tuple(map(int, input("Enter date as d m y: ").split()))
    if is_valid(date):
        d, m, y = date
        print("Valid:", "%02d/%02d/%04d" % date, "|", d, MONTHS[m - 1], y)
```

```
    else:
        print("Invalid date:", date)
```

Output:

```
Enter date as d m y: 29 2 2024
Valid: 29/02/2024 | 29 February 2024
Enter date as d m y: 31 4 2025
Invalid date: (31, 4, 2025)
Enter date as d m y: 7 10 2026
Valid: 07/10/2026 | 7 October 2026
```

Set Operations and Membership Testing

Program 3.34: Set Operations using Methods and Operators

Aim: To write a Python program that performs union, intersection, difference and symmetric difference of two sets.

Program:

```
# Basic set operations
set_a = set(map(int, input("Enter elements of set A: ").split()))
set_b = set(map(int, input("Enter elements of set B: ").split()))

print("A =", sorted(set_a))
print("B =", sorted(set_b))
print("Union        :", sorted(set_a.union(set_b)), "|", sorted(set_a | set_b))
print("Intersection :", sorted(set_a.intersection(set_b)), "|",
      sorted(set_a & set_b))
print("A - B        :", sorted(set_a.difference(set_b)), "|",
      sorted(set_a - set_b))
print("Symmetric diff:", sorted(set_a.symmetric_difference(set_b)), "|",
      sorted(set_a ^ set_b))
```

Output:

```
Enter elements of set A: 1 2 3 4 5
Enter elements of set B: 4 5 6 7
A = [1, 2, 3, 4, 5]
B = [4, 5, 6, 7]
Union        : [1, 2, 3, 4, 5, 6, 7] | [1, 2, 3, 4, 5, 6, 7]
Intersection : [4, 5] | [4, 5]
A - B        : [1, 2, 3] | [1, 2, 3]
Symmetric diff: [1, 2, 3, 6, 7] | [1, 2, 3, 6, 7]
```

Program 3.35: Subset, Superset and Disjoint Sets

Aim: To write a Python program that checks subset, superset and disjoint relationships between sets.

Program:

```
# Subset, superset and disjoint checks
set_a = set(input("Enter elements of set A: ").split())
set_b = set(input("Enter elements of set B: ").split())
set_c = set(input("Enter elements of set C: ").split())
```

```
print("A is subset of B   :", set_a.issubset(set_b), set_a <= set_b)
print("B is superset of A :", set_b.issuperset(set_a), set_b >= set_a)
print("A and C disjoint   :", set_a.isdisjoint(set_c))
print("B and C disjoint   :", set_b.isdisjoint(set_c))
```

Output:

```
Enter elements of set A: a b
Enter elements of set B: a b c d
Enter elements of set C: x y c
A is subset of B   : True True
B is superset of A : True True
A and C disjoint   : True
B and C disjoint   : False
```

Program 3.36: Remove Duplicates and Common Elements using Sets

Aim: To write a Python program that removes duplicates from a list and finds common elements of two lists using sets.

Program:

```
# Using sets on lists
list1 = list(map(int, input("Enter elements of list 1: ").split()))
list2 = list(map(int, input("Enter elements of list 2: ").split()))

unique1 = sorted(set(list1))
print("List 1 without duplicates:", unique1)

common = sorted(set(list1) & set(list2))
if common:
    print("Common elements:", common)
else:
    print("No common elements")
print("Elements only in list 1:", sorted(set(list1) - set(list2)))
```

Output:

```
Enter elements of list 1: 5 3 8 3 9 5 1
Enter elements of list 2: 8 2 5 7 5
List 1 without duplicates: [1, 3, 5, 8, 9]
Common elements: [5, 8]
Elements only in list 1: [1, 3, 9]
```

Program 3.37: Membership Testing and Frozenset

Aim: To write a Python program that demonstrates membership testing with in / not in, set modification methods and frozenset.

Program:

```
# Membership testing, set methods and frozenset
vowels = {"a", "e", "i", "o", "u"}
letter = input("Enter a letter: ").lower()
if letter in vowels:
    print(letter, "is a vowel")
```

```
else:
    print(letter, "is not a vowel")

subjects = set(input("Enter subjects: ").split())
subjects.add("Python")
subjects.discard("Drawing")      # no error even if absent
print("Subjects:", sorted(subjects))
print("Is Maths not in subjects?", "Maths" not in subjects)

fixed = frozenset(subjects)
try:
    fixed.add("Chemistry")
except AttributeError as error:
    print("Frozenset cannot be changed:", error)
print("Frozenset:", sorted(fixed))
```

Output:

```
Enter a letter: E
e is a vowel
Enter subjects: Physics Maths Drawing
Subjects: ['Maths', 'Physics', 'Python']
Is Maths not in subjects? False
Frozenset cannot be changed: 'frozenset' object has no attribute 'add'
Frozenset: ['Maths', 'Physics', 'Python']
```

Program 3.38: Unique Words in a Sentence

Aim: To write a Python program that finds the unique words in a sentence and the words common to two sentences.

Program:

```
# Unique words using sets
sentence1 = input("Enter first sentence: ").lower().split()
sentence2 = input("Enter second sentence: ").lower().split()

unique1 = set(sentence1)
unique2 = set(sentence2)
print("Total words in first sentence:", len(sentence1))
print("Unique words:", sorted(unique1))
print("Number of unique words:", len(unique1))
print("Words common to both sentences:", sorted(unique1 & unique2))
```

Output:

```
Enter first sentence: the cat sat on the mat and the cat slept
Enter second sentence: the dog sat near the cat
Total words in first sentence: 10
Unique words: ['and', 'cat', 'mat', 'on', 'sat', 'slept', 'the']
Number of unique words: 7
Words common to both sentences: ['cat', 'sat', 'the']
```

Program 3.39: Attendance - Present and Absent Students

Aim: To write a Python program that finds absent students by comparing the class roll list with the roll numbers marked present.

Program:

```
# Attendance using sets
class_rolls = set(range(1, int(input("Enter class strength: ")) + 1))
present = set(map(int, input("Enter roll numbers present: ").split()))

invalid = present - class_rolls
present = present & class_rolls
absent = class_rolls - present

print("Present (%d):" % len(present), sorted(present))
print("Absent  (%d):" % len(absent), sorted(absent))
if invalid:
    print("Invalid roll numbers ignored:", sorted(invalid))
print("Attendance: %.1f%%" % (len(present) * 100 / len(class_rolls)))
```

Output:

```
Enter class strength: 12
Enter roll numbers present: 1 2 4 5 7 8 9 11 12 15 4
Present (9): [1, 2, 4, 5, 7, 8, 9, 11, 12]
Absent  (3): [3, 6, 10]
Invalid roll numbers ignored: [15]
Attendance: 75.0%
```

Program 3.40: Course Enrollment Analysis using Sets

Aim: To write a Python program that finds students enrolled in both courses, either course and only one course.

Program:

```
# Students enrolled in Python and Java electives
python_course = set(input("Students in Python: ").split())
java_course = set(input("Students in Java: ").split())

print("Both courses   :", sorted(python_course & java_course))
print("Either course  :", sorted(python_course | java_course))
print("Only Python    :", sorted(python_course - java_course))
print("Only Java      :", sorted(java_course - python_course))
print("Exactly one    :", sorted(python_course ^ java_course))
print("Total distinct students:", len(python_course | java_course))
```

Output:

```
Students in Python: Arun Bala Chandu Divya Esha
Students in Java: Chandu Esha Farhan Gita
Both courses   : ['Chandu', 'Esha']
Either course  : ['Arun', 'Bala', 'Chandu', 'Divya', 'Esha', 'Farhan', 'Gita']
Only Python    : ['Arun', 'Bala', 'Divya']
Only Java      : ['Farhan', 'Gita']
Exactly one    : ['Arun', 'Bala', 'Divya', 'Farhan', 'Gita']
Total distinct students: 7
```

Program 3.41: Unique Visitors from a Visitor Log

Aim: To write a Python program that counts total visits and unique visitors from a website/library visitor log.

Program:

```
# Unique visitors using a set
log = input("Enter visitor IDs in order of visit: ").split()

unique = set(log)
print("Total visits   :", len(log))
print("Unique visitors:", len(unique))
print("Visitor IDs    :", sorted(unique))

repeat = sorted({v for v in log if log.count(v) > 1})
print("Repeat visitors:", repeat)
```

Output:

```
Enter visitor IDs in order of visit: U12 U07 U12 U33 U07 U12 U50 U21 U33
Total visits   : 9
Unique visitors: 5
Visitor IDs    : ['U07', 'U12', 'U21', 'U33', 'U50']
Repeat visitors: ['U07', 'U12', 'U33']
```

Program 3.42: Banned Word Filter using Set Membership

Aim: To write a Python program that masks banned words in a comment using fast set membership testing.

Program:

```
# Comment filter with a set of banned words
banned = {"stupid", "idiot", "useless", "dumb"}
comment = input("Enter a comment: ")

clean_words = []
found = set()
for word in comment.split():
    if word.lower().strip(".,!?") in banned:
        found.add(word.lower().strip(".,!?"))
        clean_words.append("*" * len(word))
    else:
        clean_words.append(word)

print("Filtered comment:", " ".join(clean_words))
print("Banned words found:", sorted(found) if found else "None")
```

Output:

```
Enter a comment: This app is useless and the update is Stupid but support helped
Filtered comment: This app is ******* and the update is ****** but support helped
Banned words found: ['stupid', 'useless']
```

Program 3.43: Common Friends and Friend Suggestions

Aim: To write a Python program that finds mutual friends of two users and suggests new friends using set operations.

Program:

```
# Mutual friends using sets
user_a = input("Name of user A: ")
friends_a = set(input("Friends of " + user_a + ": ").split())
user_b = input("Name of user B: ")
friends_b = set(input("Friends of " + user_b + ": ").split())

mutual = friends_a & friends_b
print("Mutual friends:", sorted(mutual), "(" + str(len(mutual)) + ")")

suggest_a = friends_b - friends_a - {user_a}
suggest_b = friends_a - friends_b - {user_b}
print("Suggestions for", user_a, ":", sorted(suggest_a))
print("Suggestions for", user_b, ":", sorted(suggest_b))
print("Are they friends?", user_b in friends_a)
```

Output:

```
Name of user A: Priya
Friends of Priya: Ram Kavya Joel Neha Ravi
Name of user B: Ravi
Friends of Ravi: Kavya Priya Neha Omar
Mutual friends: ['Kavya', 'Neha'] (2)
Suggestions for Priya : ['Omar']
Suggestions for Ravi : ['Joel', 'Ram']
Are they friends? True
```

Program 3.44: Check Duplicates in a List using a Set

Aim: To write a Python program that checks whether a list of registration numbers contains duplicates and lists them.

Program:

```
# Detect duplicate registration numbers
reg_numbers = input("Enter registration numbers: ").split()

if len(set(reg_numbers)) == len(reg_numbers):
    print("All registration numbers are unique")
else:
    seen = set()
    duplicates = set()
    for reg in reg_numbers:
        if reg in seen:
            duplicates.add(reg)
        else:
            seen.add(reg)
    print("Duplicates found:", sorted(duplicates))
    print("Unique entries  :", len(seen), "of", len(reg_numbers))
```

Output:

```
Enter registration numbers: 22A101 22A102 22A105 22A102 22A110 22A101 22A120
Duplicates found: ['22A101', '22A102']
Unique entries  : 5 of 7
```

Program 3.45: Common Letters and Pangram Check using Sets

Aim: To write a Python program that finds letters common to two words and checks whether a sentence is a pangram, listing missing letters.

Program:

```
# Letters in two words and pangram check
import string

word1 = input("Enter first word: ").lower()
word2 = input("Enter second word: ").lower()
print("Common letters :", sorted(set(word1) & set(word2)))
print("Only in", word1, ":", sorted(set(word1) - set(word2)))

alphabet = set(string.ascii_lowercase)
for _ in range(2):
    sentence = input("Enter a sentence: ").lower()
    missing = alphabet - set(sentence)
    if not missing:
        print("Pangram - uses every letter of the alphabet")
    else:
        print("Not a pangram. Missing letters:", "".join(sorted(missing)))
```

Output:

```
Enter first word: listen
Enter second word: silent
Common letters : ['e', 'i', 'l', 'n', 's', 't']
Only in listen : []
Enter a sentence: The quick brown fox jumps over the lazy dog
Pangram - uses every letter of the alphabet
Enter a sentence: Python programming is fun
Not a pangram. Missing letters: bcdejklqvwxz
```

Recursive Function Applications

Program 3.46: Factorial using Recursion

Aim: To write a Python program that finds the factorial of a number using recursion.

Program:

```
# Factorial using recursion
def factorial(n):
    """Return n! computed recursively."""
    if n <= 1:          # base case
        return 1
    return n * factorial(n - 1)


num = int(input("Enter a non-negative integer: "))
```

```
if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", num, "is", factorial(num))
```

Output:

```
Enter a non-negative integer: 6
Factorial of 6 is 720
```

Program 3.47: Fibonacci Series using Recursion

Aim: To write a Python program that finds the nth Fibonacci term and prints the Fibonacci series using recursion.

Program:

```
# Fibonacci using recursion
def fib(n):
    """Return the nth Fibonacci number (fib(0) = 0)."""
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)


terms = int(input("Enter number of terms: "))
print("Fibonacci series:", end=" ")
for i in range(terms):
    print(fib(i), end=" ")
print()

n = int(input("Enter n to find nth term: "))
print("Fibonacci term", n, "is", fib(n))
```

Output:

```
Enter number of terms: 10
Fibonacci series: 0 1 1 2 3 5 8 13 21 34
Enter n to find nth term: 12
Fibonacci term 12 is 144
```

Program 3.48: Sum of Digits using Recursion

Aim: To write a Python program that finds the sum of digits and number of digits of a number using recursion.

Program:

```
# Sum of digits using recursion
def sum_of_digits(n):
    """Return the sum of digits of a non-negative integer."""
    if n == 0:
        return 0
    return n % 10 + sum_of_digits(n // 10)


def count_digits(n):
```

```
    """Return the number of digits in a non-negative integer."""
    if n < 10:
        return 1
    return 1 + count_digits(n // 10)


number = abs(int(input("Enter a number: ")))
print("Sum of digits:", sum_of_digits(number))
print("Number of digits:", count_digits(number))
```

Output:

```
Enter a number: 98765
Sum of digits: 35
Number of digits: 5
```

Program 3.49: Power and GCD using Recursion

Aim: To write a Python program that computes x raised to n and the GCD of two numbers using recursion.

Program:

```
# Power and GCD using recursion
def power(x, n):
    """Return x raised to a non-negative integer n."""
    if n == 0:
        return 1
    return x * power(x, n - 1)


def gcd(a, b):
    """Return GCD of a and b using Euclid's algorithm."""
    if b == 0:
        return a
    return gcd(b, a % b)


x = int(input("Enter base x: "))
n = int(input("Enter exponent n: "))
print(x, "^", n, "=", power(x, n))

a = int(input("Enter first number for GCD: "))
b = int(input("Enter second number for GCD: "))
g = gcd(a, b)
print("GCD of", a, "and", b, "is", g)
print("LCM of", a, "and", b, "is", a * b // g)
```

Output:

```
Enter base x: 2
Enter exponent n: 10
2 ^ 10 = 1024
Enter first number for GCD: 48
Enter second number for GCD: 18
GCD of 48 and 18 is 6
LCM of 48 and 18 is 144
```

Program 3.50: Reverse String and Palindrome Check using Recursion

Aim: To write a Python program that reverses a string and checks whether it is a palindrome using recursion.

Program:

```
# Reverse string and palindrome check using recursion
def reverse(text):
    """Return text reversed."""
    if text == "":
        return ""
    return reverse(text[1:]) + text[0]


def is_palindrome(text):
    """Return True if text reads the same forwards and backwards."""
    if len(text) <= 1:
        return True
    if text[0] != text[-1]:
        return False
    return is_palindrome(text[1:-1])


word = input("Enter a string: ")
print("Reversed string:", reverse(word))
if is_palindrome(word.lower()):
    print(word, "is a palindrome")
else:
    print(word, "is not a palindrome")
```

Output:

```
Enter a string: Malayalam
Reversed string: malayalaM
Malayalam is a palindrome
```

Program 3.51: Tower of Hanoi

Aim: To write a Python program that solves the Tower of Hanoi problem using recursion.

Program:

```
# Tower of Hanoi using recursion
moves = 0


def hanoi(n, source, auxiliary, destination):
    """Print the moves to transfer n disks from source to destination."""
    global moves
    if n == 0:
        return
    hanoi(n - 1, source, destination, auxiliary)
    moves += 1
    print("Move disk", n, "from", source, "to", destination)
    hanoi(n - 1, auxiliary, source, destination)
```

```
disks = int(input("Enter number of disks: "))
hanoi(disks, "A", "B", "C")
print("Total moves:", moves)
```

Output:

```
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
```

Program 3.52: Binary Search using Recursion

Aim: To write a Python program that searches an element in a sorted list using recursive binary search.

Program:

```
# Recursive binary search
def binary_search(data, key, low, high):
    """Return index of key in sorted data[low..high], or -1."""
    if low > high:
        return -1
    mid = (low + high) // 2
    if data[mid] == key:
        return mid
    if key < data[mid]:
        return binary_search(data, key, low, mid - 1)
    return binary_search(data, key, mid + 1, high)


numbers = sorted(map(int, input("Enter numbers separated by space: ").split()))
key = int(input("Enter element to search: "))
print("Sorted list:", numbers)

index = binary_search(numbers, key, 0, len(numbers) - 1)
if index == -1:
    print(key, "not found")
else:
    print(key, "found at index", index)
```

Output:

```
Enter numbers separated by space: 45 12 78 34 89 23 56
Enter element to search: 56
Sorted list: [12, 23, 34, 45, 56, 78, 89]
56 found at index 4
```

Program 3.53: Decimal to Binary using Recursion

Aim: To write a Python program that converts a decimal number to binary using recursion.

Program:

```
# Decimal to binary conversion using recursion
def to_binary(n):
    """Return the binary representation of n as a string."""
    if n <= 1:
        return str(n)
    return to_binary(n // 2) + str(n % 2)


number = int(input("Enter a non-negative decimal number: "))
print("Binary of", number, "is", to_binary(number))
print("Check with bin():", bin(number)[2:])
```

Output:

```
Enter a non-negative decimal number: 45
Binary of 45 is 101101
Check with bin(): 101101
```

Program 3.54: Sum and Maximum of a List using Recursion

Aim: To write a Python program that finds the sum and the maximum element of a list using recursive functions.

Program:

```
# Recursive sum and maximum of a list
def list_sum(items):
    if not items:                 # base case: empty list
        return 0
    return items[0] + list_sum(items[1:])


def list_max(items):
    if len(items) == 1:           # base case: single element
        return items[0]
    rest_max = list_max(items[1:])
    return items[0] if items[0] > rest_max else rest_max


values = list(map(int, input("Enter numbers: ").split()))
print("Sum of elements :", list_sum(values))
print("Maximum element :", list_max(values))
```

Output:

```
Enter numbers: 45 12 78 3 66 91 20
Sum of elements : 315
Maximum element : 91
```

Program 3.55: Count Digits of a Number using Recursion

Aim: To write a Python program that counts the digits of a number and the occurrences of a given digit using recursion.

Program:

```
# Counting digits recursively
def count_digits(n):
    if n < 10:
        return 1
    return 1 + count_digits(n // 10)


def count_occurrence(n, d):
    if n == 0:
        return 0
    return (1 if n % 10 == d else 0) + count_occurrence(n // 10, d)


number = abs(int(input("Enter a number: ")))
digit = int(input("Enter digit to count: "))
print("Number of digits:", count_digits(number))
print("Digit", digit, "occurs", count_occurrence(number, digit), "time(s)")
```

Output:

```
Enter a number: 9073507
Enter digit to count: 7
Number of digits: 7
Digit 7 occurs 2 time(s)
```

Program 3.56: Print 1 to N and N to 1 using Recursion

Aim: To write a Python program that prints numbers from 1 to n and from n to 1 using recursion without loops.

Program:

```
# Ascending and descending order using recursion
def print_ascending(n):
    if n == 0:
        return
    print_ascending(n - 1)
    print(n, end=" ")


def print_descending(n):
    if n == 0:
        return
    print(n, end=" ")
    print_descending(n - 1)


n = int(input("Enter n: "))
```

```
print("1 to n:", end=" ")
print_ascending(n)
print()
print("n to 1:", end=" ")
print_descending(n)
print()
```

Output:

```
Enter n: 10
1 to n: 1 2 3 4 5 6 7 8 9 10
n to 1: 10 9 8 7 6 5 4 3 2 1
```

Program 3.57: Check if a List is Sorted using Recursion

Aim: To write a Python program that checks recursively whether a list is sorted in ascending order.

Program:

```
# Recursive sorted check
def is_sorted(items, index=0):
    if index >= len(items) - 1:
        return True
    if items[index] > items[index + 1]:
        return False
    return is_sorted(items, index + 1)


for _ in range(2):
    values = list(map(int, input("Enter numbers: ").split()))
    if is_sorted(values):
        print(values, "is sorted in ascending order")
    else:
        print(values, "is NOT sorted")
```

Output:

```
Enter numbers: 3 7 7 12 25 40
[3, 7, 7, 12, 25, 40] is sorted in ascending order
Enter numbers: 5 9 4 11
[5, 9, 4, 11] is NOT sorted
```

Program 3.58: Flatten a Nested List using Recursion

Aim: To write a Python program that converts a nested list of any depth into a single flat list using recursion.

Program:

```
# Flatten a nested list recursively
def flatten(items):
    flat = []
    for element in items:
        if isinstance(element, list):
            flat.extend(flatten(element))     # recursive call
        else:
            flat.append(element)
    return flat
```

```
nested = [1, [2, 3], [4, [5, 6, [7]], 8], [], 9]
print("Nested list:", nested)
print("Flat list  :", flatten(nested))

marks = [[78, 85], [90, [66, 72]], 88]
flat_marks = flatten(marks)
print("Section marks:", marks, "-> total", sum(flat_marks))
```

Output:

```
Nested list: [1, [2, 3], [4, [5, 6, [7]], 8], [], 9]
Flat list  : [1, 2, 3, 4, 5, 6, 7, 8, 9]
Section marks: [[78, 85], [90, [66, 72]], 88] -> total 479
```

Program 3.59: Permutations of a String using Recursion

Aim: To write a Python program that generates all permutations of a string using recursion.

Program:

```
# All permutations of a string
def permutations(text):
    if len(text) <= 1:
        return [text]
    result = []
    for i in range(len(text)):
        first = text[i]
        rest = text[:i] + text[i + 1:]
        for perm in permutations(rest):
            result.append(first + perm)
    return result


word = input("Enter a string: ")
perms = permutations(word)
print("Permutations:", " ".join(perms))
print("Total:", len(perms))
print("Distinct:", len(set(perms)))
```

Output:

```
Enter a string: abc
Permutations: abc acb bac bca cab cba
Total: 6
Distinct: 6
```

Program 3.60: nCr using Recursion (Pascal's Rule)

Aim: To write a Python program that computes nCr using recursion and prints Pascal's triangle.

Program:

```
# nCr using recursion
def ncr(n, r):
    if r == 0 or r == n:
        return 1
```

```
    return ncr(n - 1, r - 1) + ncr(n - 1, r)


n = int(input("Enter n: "))
r = int(input("Enter r: "))
if 0 <= r <= n:
    print("C(%d, %d) = %d" % (n, r, ncr(n, r)))
else:
    print("r must be between 0 and n")

rows = int(input("Rows of Pascal's triangle: "))
for i in range(rows):
    print(" " * (rows - i - 1) * 2, end="")
    for j in range(i + 1):
        print("%4d" % ncr(i, j), end="")
    print()
```

Output:

```
Enter n: 7
Enter r: 3
C(7, 3) = 35
Rows of Pascal's triangle: 6
             1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10   5   1
```

User Defined Functions

Program 3.61: Positional, Keyword and Default Arguments with Docstrings

Aim: To write a Python program that demonstrates positional, keyword and default arguments and documents a function with a docstring.

Program:

```
# Types of function arguments
def simple_interest(principal, years, rate=8.5):
    """Return simple interest for principal, years and rate (default 8.5%)."""
    return principal * years * rate / 100


p = float(input("Enter principal: "))
t = float(input("Enter time in years: "))
r = float(input("Enter rate of interest: "))

print("Positional arguments   :", simple_interest(p, t, r))
print("Keyword arguments      :", simple_interest(rate=r, years=t, principal=p))
print("Using default rate 8.5 :", simple_interest(p, t))
print("Docstring:", simple_interest.__doc__)
```

Output:

```
Enter principal: 10000
Enter time in years: 2
Enter rate of interest: 7
Positional arguments   : 1400.0
Keyword arguments      : 1400.0
Using default rate 8.5 : 1700.0
Docstring: Return simple interest for principal, years and rate (default 8.5%).
```

*Program 3.62: Variable Length Arguments *args and **kwargs*

Aim: To write a Python program that uses *args and **kwargs to accept a variable number of arguments.

Program:

```python
# *args and **kwargs
def total(*args):
    """Return the sum of any number of values."""
    print("args received as tuple:", args)
    result = 0
    for value in args:
        result += value
    return result


def student_profile(name, **kwargs):
    """Print a student's name followed by any extra details."""
    print("Profile of", name)
    for key, value in kwargs.items():
        print("  ", key, ":", value)


numbers = list(map(int, input("Enter numbers separated by space: ").split()))
print("Sum =", total(*numbers))
print("Sum of 1, 2, 3 =", total(1, 2, 3))

name = input("Enter student name: ")
student_profile(name, branch="CSE", year=1, section="A")
```

Output:

```
Enter numbers separated by space: 5 10 15 20
args received as tuple: (5, 10, 15, 20)
Sum = 50
args received as tuple: (1, 2, 3)
Sum of 1, 2, 3 = 6
Enter student name: Kavya
Profile of Kavya
  branch : CSE
  year : 1
  section : A
```

Program 3.63: Function Returning Multiple Values

Aim: To write a Python program in which a function returns the minimum, maximum and average of a list.

Program:

```
# Returning multiple values from a function
def statistics(values):
    """Return minimum, maximum and average of a list."""
    smallest = min(values)
    largest = max(values)
    average = sum(values) / len(values)
    return smallest, largest, average


marks = list(map(int, input("Enter marks separated by space: ").split()))
low, high, avg = statistics(marks)
print("Minimum:", low)
print("Maximum:", high)
print("Average: %.2f" % avg)
print("Returned object:", statistics(marks))
```

Output:

```
Enter marks separated by space: 67 89 45 92 76
Minimum: 45
Maximum: 92
Average: 73.80
Returned object: (45, 92, 73.8)
```

Program 3.64: Global and Local Scope and Closures

Aim: To write a Python program that demonstrates local and global variables, the global keyword and a nested function (closure).

Program:

```
# Scope of variables and closures
counter = 0          # global variable


def show_local():
    counter = 100    # local variable, hides the global one
    print("Inside show_local, counter =", counter)


def increment():
    global counter
    counter += 1


def make_multiplier(factor):
    """Return a function that multiplies its argument by factor."""
    def multiply(x):
```

```
        return x * factor     # factor is remembered (closure)
    return multiply


show_local()
print("Global counter before increment:", counter)
times = int(input("How many times to increment? "))
for _ in range(times):
    increment()
print("Global counter after increment:", counter)

factor = int(input("Enter multiplication factor: "))
multiplier = make_multiplier(factor)
print("Multiplier applied to 1 to 5:", [multiplier(i) for i in range(1, 6)])
```

Output:

```
Inside show_local, counter = 100
Global counter before increment: 0
How many times to increment? 3
Global counter after increment: 3
Enter multiplication factor: 7
Multiplier applied to 1 to 5: [7, 14, 21, 28, 35]
```

Program 3.65: Lambda with map, filter, reduce and sorted

Aim: To write a Python program that uses lambda functions with map(), filter(), reduce() and sorted().

Program:

```
# Lambda functions with map, filter, reduce and sorted
from functools import reduce

numbers = list(map(int, input("Enter numbers separated by space: ").split()))

squares = list(map(lambda x: x ** 2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
product = reduce(lambda a, b: a * b, numbers)
largest = reduce(lambda a, b: a if a > b else b, numbers)

print("Squares (map)      :", squares)
print("Evens (filter)     :", evens)
print("Product (reduce)   :", product)
print("Largest (reduce)   :", largest)

words = input("Enter words separated by space: ").split()
print("Sorted by length   :", sorted(words, key=lambda w: len(w)))
print("Sorted by last char:", sorted(words, key=lambda w: w[-1]))
```

Output:

```
Enter numbers separated by space: 3 8 5 12 7 6
Squares (map)      : [9, 64, 25, 144, 49, 36]
Evens (filter)     : [8, 12, 6]
Product (reduce)   : 60480
Largest (reduce)   : 12
```

```
Enter words separated by space: banana fig apple kiwi cherry
Sorted by length   : ['fig', 'kiwi', 'apple', 'banana', 'cherry']
Sorted by last char: ['banana', 'apple', 'fig', 'kiwi', 'cherry']
```

Program 3.66: Prime Numbers using a Function

Aim: To write a Python program that defines a function to check a prime number and uses it to list all primes in a range.

Program:

```
# Prime check function reused to list primes
def is_prime(n):
    """Return True if n is a prime number."""
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True


num = int(input("Enter a number to check: "))
print(num, "is prime" if is_prime(num) else "is not prime")

low = int(input("Enter lower limit: "))
high = int(input("Enter upper limit: "))
primes = [x for x in range(low, high + 1) if is_prime(x)]
print("Primes between", low, "and", high, ":", primes)
print("Count:", len(primes))
```

Output:

```
Enter a number to check: 29
29 is prime
Enter lower limit: 10
Enter upper limit: 50
Primes between 10 and 50 : [11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
Count: 11
```

Program 3.67: Menu-Driven Calculator using Functions

Aim: To write a Python program that implements a menu-driven calculator using separate functions for each operation.

Program:

```
# Menu-driven calculator using functions
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b
```

```
def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        return "Error: division by zero"
    return a / b


operations = {"1": ("Addition", add), "2": ("Subtraction", subtract),
              "3": ("Multiplication", multiply), "4": ("Division", divide)}

while True:
    print("1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit")
    choice = input("Enter your choice: ")
    if choice == "5":
        print("Exiting calculator")
        break
    if choice not in operations:
        print("Invalid choice")
        continue
    x = float(input("Enter first number: "))
    y = float(input("Enter second number: "))
    name, function = operations[choice]
    print(name, "result:", function(x, y))
```

Output:

```
1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit
Enter your choice: 1
Enter first number: 12
Enter second number: 8
Addition result: 20.0
1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit
Enter your choice: 3
Enter first number: 4
Enter second number: 2.5
Multiplication result: 10.0
1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit
Enter your choice: 4
Enter first number: 9
Enter second number: 0
Division result: Error: division by zero
1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit
Enter your choice: 7
Invalid choice
1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit
Enter your choice: 5
Exiting calculator
```

Program 3.68: Menu-Driven Unit Converter using Functions

Aim: To write a Python program that converts length, weight and temperature units using separate functions selected from a menu.

Program:

```
# Unit converter with one function per conversion
def km_to_miles(km):
    return km * 0.621371


def kg_to_pounds(kg):
    return kg * 2.20462


def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32


def inches_to_cm(inches):
    return inches * 2.54


while True:
    print("1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit")
    choice = input("Enter choice: ")
    if choice == "5":
        print("Converter closed")
        break
    if choice not in ("1", "2", "3", "4"):
        print("Invalid choice")
        continue
    value = float(input("Enter value: "))
    if choice == "1":
        print("%.2f km = %.2f miles" % (value, km_to_miles(value)))
    elif choice == "2":
        print("%.2f kg = %.2f pounds" % (value, kg_to_pounds(value)))
    elif choice == "3":
        print("%.2f C = %.2f F" % (value, celsius_to_fahrenheit(value)))
    else:
        print("%.2f in = %.2f cm" % (value, inches_to_cm(value)))
```

Output:

```
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 1
Enter value: 42.195
42.20 km = 26.22 miles
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 2
Enter value: 65
65.00 kg = 143.30 pounds
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 3
Enter value: 37
```

```
37.00 C = 98.60 F
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 4
Enter value: 12
12.00 in = 30.48 cm
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 9
Invalid choice
1.km->miles 2.kg->pounds 3.C->F 4.inches->cm 5.Exit
Enter choice: 5
Converter closed
```

Program 3.69: Mean, Median and Mode using Functions

Aim: To write a Python program that computes the mean, median and mode of a data set using user-defined functions.

Program:

```python
# Basic statistics using user-defined functions
def mean(data):
    return sum(data) / len(data)


def median(data):
    s = sorted(data)
    n = len(s)
    mid = n // 2
    if n % 2 == 1:
        return s[mid]
    return (s[mid - 1] + s[mid]) / 2


def mode(data):
    """Return a list of the most frequent value(s)."""
    highest = max(data.count(x) for x in data)
    return sorted({x for x in data if data.count(x) == highest})


scores = list(map(int, input("Enter scores: ").split()))
print("Data  :", scores)
print("Mean  : %.2f" % mean(scores))
print("Median:", median(scores))
print("Mode  :", mode(scores))
```

Output:

```
Enter scores: 72 85 90 85 66 78 90 85 70 64
Data  : [72, 85, 90, 85, 66, 78, 90, 85, 70, 64]
Mean  : 78.50
Median: 81.5
Mode  : [85]
```

Program 3.70: Number Utility Functions - Armstrong, Palindrome, Perfect

Aim: To write a Python program that defines reusable functions is_armstrong(), is_palindrome() and is_perfect() and uses them together to classify numbers in a range.

Program:

```
# Reusable number-checking functions
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    return n == sum(int(d) ** power for d in digits)


def is_palindrome(n):
    return str(n) == str(n)[::-1]


def is_perfect(n):
    if n < 2:
        return False
    return n == sum(d for d in range(1, n // 2 + 1) if n % d == 0)


num = int(input("Enter a number to test: "))
print(num, "-> Armstrong:", is_armstrong(num), "| Palindrome:",
      is_palindrome(num), "| Perfect:", is_perfect(num))

low, high = map(int, input("Enter range (low high): ").split())
print("Armstrong numbers :", [n for n in range(low, high + 1) if is_armstrong(n)])
print("Perfect numbers   :", [n for n in range(low, high + 1) if is_perfect(n)])
print("Palindromes > 100 :",
      [n for n in range(max(low, 101), high + 1) if is_palindrome(n)][:10], "...")
```

Output:

```
Enter a number to test: 153
153 -> Armstrong: True | Palindrome: False | Perfect: False
Enter range (low high): 1 500
Armstrong numbers : [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Perfect numbers   : [6, 28, 496]
Palindromes > 100 : [101, 111, 121, 131, 141, 151, 161, 171, 181, 191] ...
```

Program 3.71: Simple Interest and Loan EMI with Default Arguments

Aim: To write a Python program that computes simple interest and the monthly EMI of a loan using functions with default arguments.

Program:

```
# Simple interest and EMI with default arguments
def simple_interest(principal, rate=8.5, years=1):
    return principal * rate * years / 100
```

```
def emi(principal, annual_rate=9.0, months=12):
    r = annual_rate / (12 * 100)
    factor = (1 + r) ** months
    return principal * r * factor / (factor - 1)


p = float(input("Enter principal amount: "))
rate = float(input("Enter annual rate (%): "))
years = int(input("Enter time in years: "))

print("SI with defaults (8.5%%, 1 yr): %.2f" % simple_interest(p))
print("SI with given values        : %.2f" % simple_interest(p, rate, years))

monthly = emi(p, rate, years * 12)
print("Monthly EMI                 : %.2f" % monthly)
print("Total payment               : %.2f" % (monthly * years * 12))
print("Total interest              : %.2f" % (monthly * years * 12 - p))
print("EMI with defaults (9%%, 12 m): %.2f" % emi(p))
```

Output:

```
Enter principal amount: 500000
Enter annual rate (%): 10.5
Enter time in years: 5
SI with defaults (8.5%, 1 yr): 42500.00
SI with given values        : 262500.00
Monthly EMI                 : 10746.95
Total payment               : 644817.01
Total interest              : 144817.01
EMI with defaults (9%, 12 m): 43725.74
```

Program 3.72: GST Bill Generator using Functions

Aim: To write a Python program that generates a GST invoice where functions compute line amounts and return multiple values (subtotal, CGST, SGST, grand total).

Program:

```
# GST bill using functions returning multiple values
def item_amount(price, qty):
    return price * qty


def compute_gst(amount, gst_rate):
    cgst = amount * gst_rate / 200
    sgst = amount * gst_rate / 200
    return cgst, sgst, amount + cgst + sgst


def print_line(name, qty, price, rate, total):
    print("%-10s %3d %8.2f %4d%% %10.2f" % (name, qty, price, rate, total))


n = int(input("Number of items: "))
subtotal = total_cgst = total_sgst = 0
```

```
lines = []
for _ in range(n):
    name, qty, price, rate = input("name qty price gst%: ").split()
    qty, price, rate = int(qty), float(price), int(rate)
    amount = item_amount(price, qty)
    cgst, sgst, total = compute_gst(amount, rate)       # unpacking
    subtotal += amount
    total_cgst += cgst
    total_sgst += sgst
    lines.append((name, qty, price, rate, total))

print("%-10s %3s %8s %5s %10s" % ("Item", "Qty", "Price", "GST", "Amount"))
for line in lines:
    print_line(*line)
print("Subtotal   : %.2f" % subtotal)
print("CGST       : %.2f" % total_cgst)
print("SGST       : %.2f" % total_sgst)
print("Grand total: %.2f" % (subtotal + total_cgst + total_sgst))
```

Output:

```
Number of items: 3
name qty price gst%: Shirt 2 799 12
name qty price gst%: Shoes 1 2499 18
name qty price gst%: Rice 5 62 5
Item       Qty    Price   GST     Amount
Shirt        2   799.00   12%    1789.76
Shoes        1  2499.00   18%    2948.82
Rice         5    62.00    5%     325.50
Subtotal   : 4407.00
CGST       : 328.54
SGST       : 328.54
Grand total: 5064.08
```

Program 3.73: Password Validation Function Returning Reasons

Aim: To write a Python program with a function that validates a password and returns the list of rules it fails.

Program:

```
# Password validator returning a list of reasons
def validate_password(pwd):
    reasons = []
    if len(pwd) < 8:
        reasons.append("must be at least 8 characters")
    if not any(ch.isupper() for ch in pwd):
        reasons.append("needs an uppercase letter")
    if not any(ch.islower() for ch in pwd):
        reasons.append("needs a lowercase letter")
    if not any(ch.isdigit() for ch in pwd):
        reasons.append("needs a digit")
    if not any(ch in "!@#$%^&*_-" for ch in pwd):
        reasons.append("needs a special character (!@#$%^&*_-)")
    if " " in pwd:
        reasons.append("must not contain spaces")
    return reasons
```

```
for _ in range(3):
    password = input("Enter password: ")
    problems = validate_password(password)
    if not problems:
        print("Strong password - accepted")
    else:
        print("Rejected:")
        for reason in problems:
            print("  -", reason)
```

Output:

```
Enter password: hello
Rejected:
  - must be at least 8 characters
  - needs an uppercase letter
  - needs a digit
  - needs a special character (!@#$%^&*_-)
Enter password: Welcome2025
Rejected:
  - needs a special character (!@#$%^&*_-)
Enter password: Rvr@Cse2026
Strong password - accepted
```

Program 3.74: Plain vs Memoized Fibonacci - Function Call Count

Aim: To write a Python program that compares the number of function calls made by plain recursive Fibonacci and memoized Fibonacci.

Program:

```
# Counting calls: plain recursion vs memoization
calls = 0


def fib_plain(n):
    global calls
    calls += 1
    if n <= 1:
        return n
    return fib_plain(n - 1) + fib_plain(n - 2)


memo = {}


def fib_memo(n):
    global calls
    calls += 1
    if n in memo:
        return memo[n]
    result = n if n <= 1 else fib_memo(n - 1) + fib_memo(n - 2)
    memo[n] = result
    return result
```

```
for n in map(int, input("Enter values of n: ").split()):
    calls = 0
    value = fib_plain(n)
    plain_calls = calls

    calls = 0
    memo.clear()
    fib_memo(n)
    print("fib(%d) = %d | plain calls = %d | memoized calls = %d"
          % (n, value, plain_calls, calls))
```

Output:

```
Enter values of n: 10 20 25
fib(10) = 55 | plain calls = 177 | memoized calls = 19
fib(20) = 6765 | plain calls = 21891 | memoized calls = 39
fib(25) = 75025 | plain calls = 242785 | memoized calls = 49
```